AURORA TICKETS

EN ESTA MEMORIA VAMOS A APRECIAR EL PASO A PASO A MODO DE TUTORIALES PARA REALIZAR LA PRÁCTICA CON ÉXITO.

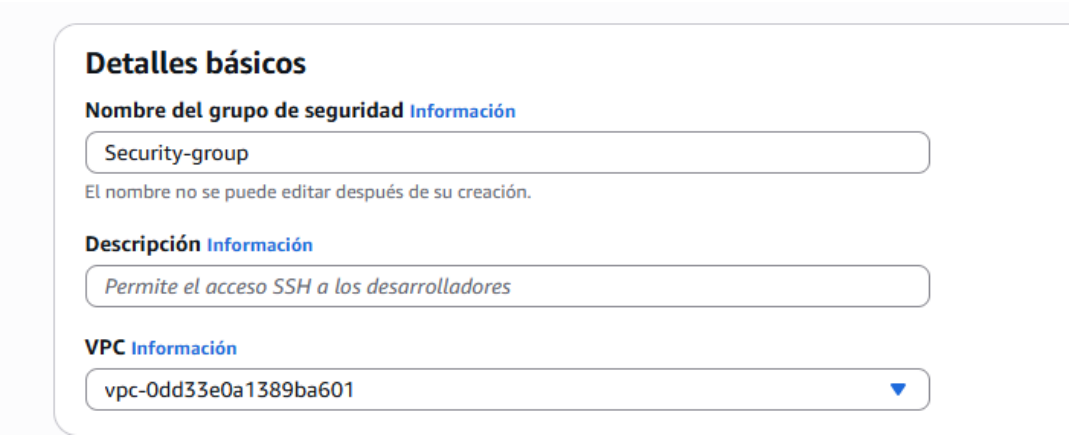
ÍNDICE

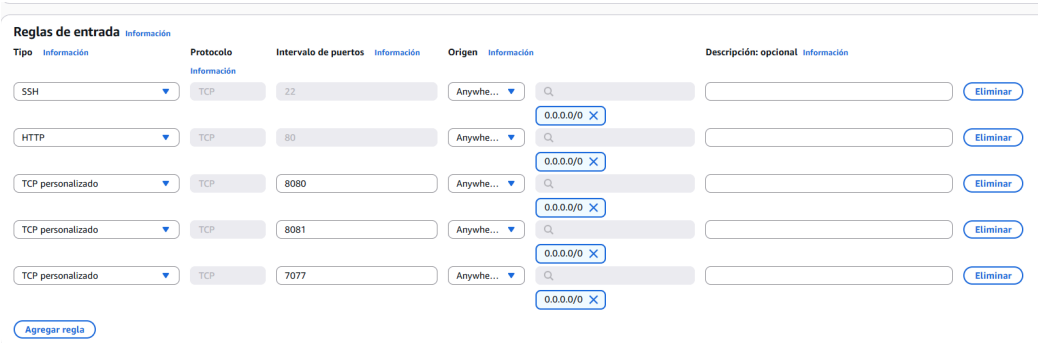
1. Configurar el Security Group 2
2. Levantar el nodo principal, Spark Master (EC2-1) y desplegar los 3 Spark Workers. 3
3. Desplegar Nodo Submit (EC2-5) y desplegar Nodo Web (EC2-6) 9
4. Web y Clickstream. 12
5. Salto a CloudWatch 27
6. Creación de Bucket y Jobs. 32
7. CloudWatch (Insights y Dashboards) 49
8. Costes 55

1. Configurar el Security Group

Hacemos clic en el botón **agregar regla** cinco veces y configuramos cada línea así:

1. **Tipo:** SSH | **Puerto:** 22 | **Origen:** `0.0.0.0/0` (Para entrar por terminal a las máquinas).
2. **Tipo:** HTTP | **Puerto:** 80 | **Origen:** `0.0.0.0/0` (Para acceder a la web).
3. **Tipo:** TCP personalizado | **Puerto:** 8080 | **Origen:** `0.0.0.0/0` (Para ver el panel del Master de Spark).
4. **Tipo:** TCP personalizado | **Puerto:** 8081 | **Origen:** `0.0.0.0/0` (Para ver el panel de los Workers de Spark).
5. **Tipo:** TCP personalizado | **Puerto:** 7077 | **Origen:** `0.0.0.0/0` (Para que el Master y los Workers se comuniquen).





2. Levantar el nodo principal, Spark Master (EC2-1) y desplegar los 3 Spark Workers.

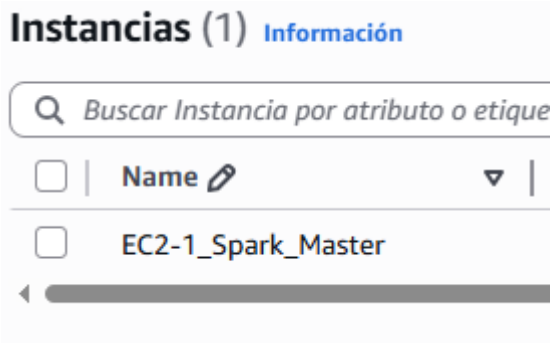
Seguimos estos pasos exactos en la consola de AWS:

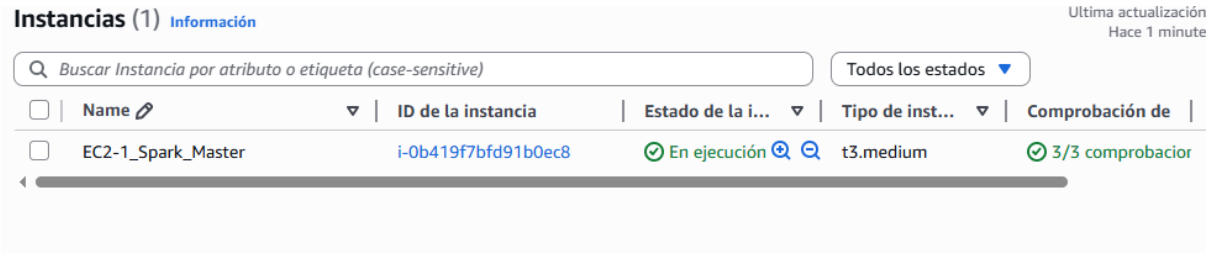
1. Ir a **EC2** y pulsar en el botón naranja **Lanzar instancia**.
2. **Nombre y etiquetas:** `EC2-1_Spark_Master`.
3. **Imágenes de SO (AMI):** Seleccionamos **Ubuntu**, es el sistema recomendado para esta práctica.
4. **Tipo de instancia:** `t3.medium`.
5. **Par de claves:** vokey
6. **Configuraciones de red:**
   - Marcar la casilla **Seleccionar un grupo de seguridad existente**.
   - Abre el desplegable y elige `Security-group` (el que acabamos de crear).
7. **Detalles avanzados:** Bajamos hasta el final del todo, donde el cuadro de texto llamado **Datos de usuario** (User data) y pegamos el siguiente código:

   (Esto instala Java, Python y la versión obligatoria de Spark de forma automática)

```
#!/bin/bash

apt-get update

apt-get install -y openjdk-11-jdk-headless wget tar python3
python3-pip

wget
https://archive.apache.org/dist/spark/spark-3.5.1/spark-3.5.
1-bin-hadoop3.tgz

tar -xzf spark-3.5.1-bin-hadoop3.tgz -C /opt/

mv /opt/spark-3.5.1-bin-hadoop3 /opt/spark

echo "export SPARK_HOME=/opt/spark" >>
/etc/profile.d/spark.sh

echo "export PATH=\$PATH:\$SPARK_HOME/bin:\$SPARK_HOME/sbin"
>> /etc/profile.d/spark.sh

source /etc/profile.d/spark.sh
```

```
/opt/spark/sbin/start-master.sh
```





Desplegar los 3 Spark Workers:

Seguimos estos pasos:

1. **Lanzar instancia**.
2. En **Número de instancias** (arriba a la derecha), escribimos **3**. Así creamos las tres máquinas de golpe.
3. **Nombre:** `EC2_Spark_Worker (luego cambiamos los nombres)`.
4. Elegimos **Ubuntu** y el mismo tipo de máquina (`t3.medium`).
5. Seleccionamos el par de claves.
6. En red, marcamos **Seleccionar un grupo de seguridad existente** y elige grupo `Security-group.`
7. En **Datos de usuario** pegamos este código. Ya tiene la IP del Master puesta en la última línea para que se conecten solos.

```
#!/bin/bash
apt-get update
apt-get install -y openjdk-11-jdk-headless wget tar
python3 python3-pip
wget
https://archive.apache.org/dist/spark/spark-3.5.1/spar
k-3.5.1-bin-hadoop3.tgz
tar -xzf spark-3.5.1-bin-hadoop3.tgz -C /opt/
mv /opt/spark-3.5.1-bin-hadoop3 /opt/spark
echo "export SPARK_HOME=/opt/spark" >>
/etc/profile.d/spark.sh
echo "export
PATH=\$PATH:\$SPARK_HOME/bin:\$SPARK_HOME/sbin" >>
/etc/profile.d/spark.sh
source /etc/profile.d/spark.sh
/opt/spark/sbin/start-worker.sh
spark://172.31.14.249:7077
```

8. Pulsar **Lanzar instancia**.

Mientras estas tres máquinas se inician, podemos comprobar si el Master funciona, abriendo una pestaña nueva en el navegador y pegando la IP pública del Master con el puerto 8080: `http://3.238.4.217:8080` (en este caso).

Antes de continuar vamos a comprobar que en el master está instalado Java y Spark y también que este último esté funcionando.

Escribiendo estos comandos uno a uno en esa pantalla negra y dándole a Enter.

1. **`java -version`** *(Para ver si Java se instaló correctamente. Es imprescindible para Spark).*

2. **`ps aux | grep spark`** *(Para comprobar si el proceso de Spark está funcionando en segundo plano).*

```
Welcome to Ubuntu 24.04.3 LTS (GNU/Linux 6.14.0-1018-aws x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:     https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

 System information as of Thu Mar 19 18:30:49 UTC 2026

  System load:  0.0               Temperature:           -273.1 C
  Usage of /:   58.0% of 6.71GB   Processes:             113
  Memory usage: 9%                Users logged in:       0
  Swap usage:   0%                IPv4 address for ens5: 172.31.14.249

Expanded Security Maintenance for Applications is not enabled.

41 updates can be applied immediately.
To see these additional updates run: apt list --upgradable

2 additional security updates can be applied with ESM Apps.
Learn more about enabling ESM Apps service at https://ubuntu.com/esm


*** System restart required ***

The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-14-249:~$ java -version
openjdk version "11.0.30" 2026-01-20
OpenJDK Runtime Environment (build 11.0.30+7-post-Ubuntu-1ubuntu124.04)
OpenJDK 64-Bit Server VM (build 11.0.30+7-post-Ubuntu-1ubuntu124.04, mixed mode, sharing)
ubuntu@ip-172-31-14-249:~$
```

```
Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

ubuntu@ip-172-31-14-249:~$ java -version
openjdk version "11.0.30" 2026-01-20
OpenJDK Runtime Environment (build 11.0.30+7-post-Ubuntu-1ubuntu124.04)
OpenJDK 64-Bit Server VM (build 11.0.30+7-post-Ubuntu-1ubuntu124.04, mixed mode, sharing)
ubuntu@ip-172-31-14-249:~$ ps aux | grep spark
ubuntu      18225  0.0  0.0   7076  2224 pts/0    S+   18:33   0:00 grep --color=auto spark
ubuntu@ip-172-31-14-249:~$
```

Java está perfectamente instalado, pero el proceso del Spark Master no está corriendo (el grep solo muestra la propia búsqueda). El script de inicio en el user-data probablemente falló o no tuvo tiempo de ejecutarse del todo.

No hay problema, vamos a levantarlo a mano ejecutando este comando en esa misma consola:

```
sudo /opt/spark/sbin/start-master.sh -h 0.0.0.0
```

Ese -h 0.0.0.0 le dice al Master que acepte conexiones desde cualquier IP.

Después de lanzarlo, volvemos a probar con http://3.238.4.217:8080 en el navegador.

Pasos para arrancar los Workers:

- Ir a la consola de AWS y seleccionar el primer **EC2_Spark_Worker**.
- Darle a **Conectar** para abrir su terminal negra.
- Ejecutar este código exacto para engancharlo al Master:

1. Escribir este comando para detener el trabajador actual:

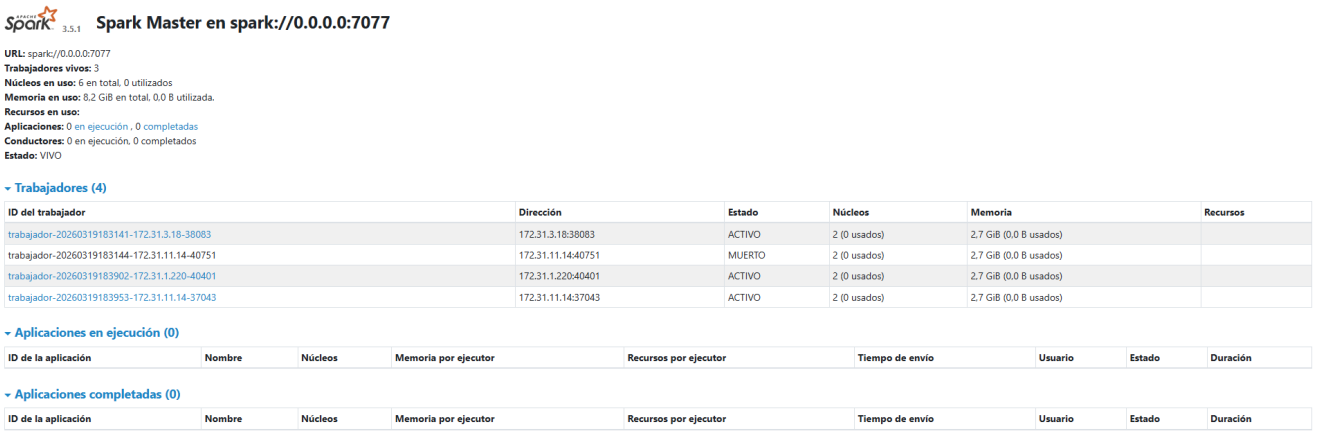
```
sudo /opt/spark/sbin/stop-worker.sh
```

2. Volver a encenderlo apuntando a la IP de nuestro Máster:

```
sudo /opt/spark/sbin/start-worker.sh
spark://172.31.14.249:7077
```

- Repetimos este mismo proceso entrando en los otros dos Workers.

Una vez lanzadas las tres máquinas, volvemos al navegador y recargamos la página web del Máster. Aparecen 3 trabajadores vivos en la lista.



Spark 3.5.1 **Spark Master en spark://0.0.0.0:7077**

URL: spark://0.0.0.0:7077
Trabajadores vivos: 3
Núcleos en uso: 6 en total, 0 utilizados
Memoria en uso: 8,2 GiB en total, 0,0 B utilizada.
Recursos en uso:
Aplicaciones: 0 en ejecución , 0 completadas
Conductores: 0 en ejecución, 0 completados
Estado: VIVO

Trabajadores (4)

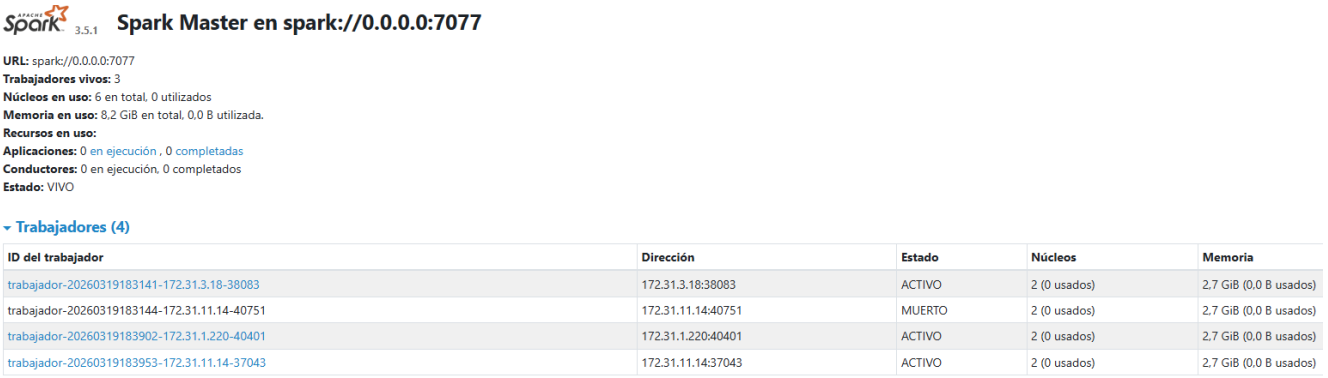
ID del trabajador	Dirección	Estado	Núcleos	Memoria	Recursos
trabajador-20260319183141-172.31.3.18-38083	172.31.3.18:38083	ACTIVO	2 (0 usados)	2,7 GiB (0,0 B usados)	
trabajador-20260319183144-172.31.11.14-40751	172.31.11.14:40751	MUERTO	2 (0 usados)	2,7 GiB (0,0 B usados)	
trabajador-20260319183902-172.31.1.220-40401	172.31.1.220:40401	ACTIVO	2 (0 usados)	2,7 GiB (0,0 B usados)	
trabajador-20260319183953-172.31.11.14-37043	172.31.11.14:37043	ACTIVO	2 (0 usados)	2,7 GiB (0,0 B usados)	

Aplicaciones en ejecución (0)

ID de la aplicación	Nombre	Núcleos	Memoria por ejecutor	Recursos por ejecutor	Tiempo de envío	Usuario	Estado	Duración

Aplicaciones completadas (0)

ID de la aplicación	Nombre	Núcleos	Memoria por ejecutor	Recursos por ejecutor	Tiempo de envío	Usuario	Estado	Duración



Spark 3.5.1 **Spark Master en spark://0.0.0.0:7077**

URL: spark://0.0.0.0:7077
Trabajadores vivos: 3
Núcleos en uso: 6 en total, 0 utilizados
Memoria en uso: 8,2 GiB en total, 0,0 B utilizada.
Recursos en uso:
Aplicaciones: 0 en ejecución , 0 completadas
Conductores: 0 en ejecución, 0 completados
Estado: VIVO

Trabajadores (4)

ID del trabajador	Dirección	Estado	Núcleos	Memoria
trabajador-20260319183141-172.31.3.18-38083	172.31.3.18:38083	ACTIVO	2 (0 usados)	2,7 GiB (0,0 B usados)
trabajador-20260319183144-172.31.11.14-40751	172.31.11.14:40751	MUERTO	2 (0 usados)	2,7 GiB (0,0 B usados)
trabajador-20260319183902-172.31.1.220-40401	172.31.1.220:40401	ACTIVO	2 (0 usados)	2,7 GiB (0,0 B usados)
trabajador-20260319183953-172.31.11.14-37043	172.31.11.14:37043	ACTIVO	2 (0 usados)	2,7 GiB (0,0 B usados)

3. Desplegar Nodo Submit (EC2-5) y desplegar Nodo Web (EC2-6)

1. Ir a **Lanzar instancia** en AWS.
2. **Nombre:** EC2-5_Submit.
3. Elige **Ubuntu** y tipo t3.medium.
4. Seleccionamos el par de claves y el grupo de seguridad Security-group.
5. En **Datos de usuario**, pegamos este código. Es igual que los anteriores, pero sin arrancar ningún servicio al final:

```
#!/bin/bash
apt-get update
apt-get install -y openjdk-11-jdk-headless wget tar python3
python3-pip
wget
https://archive.apache.org/dist/spark/spark-3.5.1/spark-3.5.1-bin
-hadoop3.tgz
tar -xzf spark-3.5.1-bin-hadoop3.tgz -C /opt/
mv /opt/spark-3.5.1-bin-hadoop3 /opt/spark
echo "export SPARK_HOME=/opt/spark" >> /etc/profile.d/spark.sh
echo "export PATH=\$PATH:\$SPARK_HOME/bin:\$SPARK_HOME/sbin" >>
/etc/profile.d/spark.sh
source /etc/profile.d/spark.sh
```

Instancias (5) Información

Buscar Instancia por atributo o etiqueta (case-sensitive)

Name	ID de la instancia	Estado de la i...
EC2-1_Spark_Master	i-0b419f7bfd91b0ec8	En ejecución
EC2_Spark_Worker1	i-0fcd7e40682ddbc82	En ejecución
EC2_Spark_Worker2	i-03dc1b985463a6a2b	En ejecución
EC2_Spark_Worker3	i-01a5b1af23f67787c	En ejecución
EC2-5_Submit	i-063c96118125dc70f	En ejecución

Nodo Web (EC2-6):

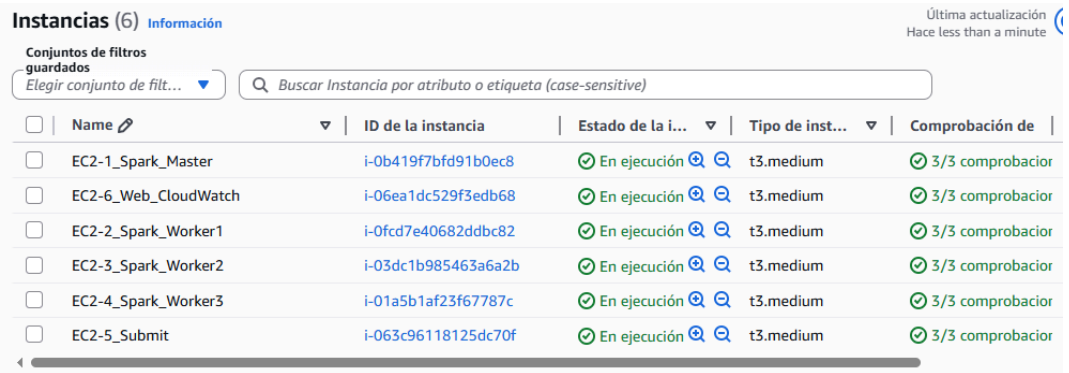
Seguimos los mismos pasos básicos en la consola de AWS:

1. Ir a **Lanzar instancia**.
2. **Nombre:** EC2-6 _Web_ CloudWatch
3. **Imágenes de SO:** Ubuntu
4. **Tipo de instancia:** La misma que las anteriores (t3.medium).
5. **Par de claves:** Par de claves habitual.
6. **Configuraciones de red:** Selecciona el grupo de seguridad Security-group (recuerdo que abrimos el puerto HTTP 80, que es obligatorio para la web).

7. **Datos de usuario (User data):** Pega este código. Esto instalará Python, Git y descargará el agente de CloudWatch, además de clonar el repositorio de GitHub que te dio el profe con el código de la web.

```
#!/bin/bash
apt-get update
# Instalar herramientas básicas y Python
apt-get install -y python3 python3-pip python3-venv git curl
wget

# Descargar el instalador del agente de CloudWatch
wget
https://s3.amazonaws.com/amazoncloudwatch-agent/ubuntu/amd64
/latest/amazon-cloudwatch-agent.deb
dpkg -i -E ./amazon-cloudwatch-agent.deb

# Clonar el repositorio del proyecto en la carpeta del
usuario ubuntu
git clone https://github.com/rjalamanac/SBD.git
/home/ubuntu/SBD
chown -R ubuntu:ubuntu /home/ubuntu/SBD
```

8. Pulsa **Lanzar instancia**.



4. Web y Clickstream.

El objetivo ahora es levantar la página web de *Aurora Tickets* en la máquina **EC2-6**, hacer que empiece a registrar las visitas (logs en formato JSON) y conectar el agente de CloudWatch para que mande esos logs a AWS .

Vamos a meterle mano a esa máquina:

1. Ir a la consola de AWS, seleccionamos la **EC2-6_Web_CloudWatch** y nos conectamos pinchando en **Conectar**.
2. En el script automático que le pusimos, le dijimos que descargara el código de github en la carpeta `/home/ubuntu/SBD`. Vamos a comprobar que está todo ahí y a ver qué ficheros hay.
3. Escribimos estos dos comandos en el terminal de la EC2-6:

```
cd /home/ubuntu/SBD

ls -la
```

```
ubuntu@ip-172-31-90-78:~$ cd /home/ubuntu/SBD
ls -la
total 36
drwxr-xr-x  7 ubuntu ubuntu 4096 Mar 24 16:25  .
drwxr-x---  5 ubuntu ubuntu 4096 Mar 24 16:32  ..
drwxr-xr-x  8 ubuntu ubuntu 4096 Mar 24 16:25  .git
-rw-r--r--  1 ubuntu ubuntu 1068 Mar 24 16:25  LICENSE
-rw-r--r--  1 ubuntu ubuntu    5 Mar 24 16:25  README.md
drwxr-xr-x  6 ubuntu ubuntu 4096 Mar 24 16:25 'Tema 1'
drwxr-xr-x  2 ubuntu ubuntu 4096 Mar 24 16:25 'Tema 2'
drwxr-xr-x 10 ubuntu ubuntu 4096 Mar 24 16:25 'Tema 3'
drwxr-xr-x  6 ubuntu ubuntu 4096 Mar 24 16:25 'Tema 4'
ubuntu@ip-172-31-90-78:~/SBD$
```

Preparar la carpeta de logs:

El proyecto exige que la web guarde los eventos en una ruta específica. Vamos a crearla y a darle permisos para que nuestro usuario no tenga problemas al escribir en ella:

```
sudo mkdir -p /var/log/aurora
sudo chown -R ubuntu:ubuntu /var/log/aurora
```

Buscar el código de la web:

Vamos a usar el buscador de Linux, para asegurarnos que el código de la web Aurora Tickets está metido en la carpeta correcta:

```
find . -iname "*aurora*" -o -iname "*fastapi*" -o -name
"requirements.txt"
```

```
ubuntu@ip-172-31-90-78:~$ cd /home/ubuntu/SBD
ls -la
total 36
drwxr-xr-x  7 ubuntu ubuntu 4096 Mar 24 16:25  .
drwxr-x---  5 ubuntu ubuntu 4096 Mar 24 16:32  ..
drwxr-xr-x  8 ubuntu ubuntu 4096 Mar 24 16:25  .git
-rw-r--r--  1 ubuntu ubuntu 1068 Mar 24 16:25  LICENSE
-rw-r--r--  1 ubuntu ubuntu    5 Mar 24 16:25  README.md
drwxr-xr-x  6 ubuntu ubuntu 4096 Mar 24 16:25 'Tema 1'
drwxr-xr-x  2 ubuntu ubuntu 4096 Mar 24 16:25 'Tema 2'
drwxr-xr-x 10 ubuntu ubuntu 4096 Mar 24 16:25 'Tema 3'
drwxr-xr-x  6 ubuntu ubuntu 4096 Mar 24 16:25 'Tema 4'
ubuntu@ip-172-31-90-78:~/SBD$ sudo mkdir -p /var/log/aurora
sudo chown -R ubuntu:ubuntu /var/log/aurora
ubuntu@ip-172-31-90-78:~/SBD$ find . -iname "*aurora*" -o -iname "*fastapi*" -o -name "requirements.txt"
./Tema 1/Docker/requirements.txt
./Tema 4/webapp/backend/requirements.txt
./Tema 4/webapp/deploy/nginx/aurora.nginx.conf.template
./Tema 4/webapp/deploy/systemd/aurora-backend.service
./Tema 4/webapp/deploy/aurora.env.template
ubuntu@ip-172-31-90-78:~/SBD$
```

El código de la web de Aurora Tickets se ha metido en la ruta `Tema 4/webapp/`.

Vamos a preparar el entorno de Python para arrancar el backend (FastAPI).

Seguimos estos pasos exactos en tu terminal:

1. Entrar en la carpeta del backend:

```
cd "/home/ubuntu/SBD/Tema 4/webapp/backend"
```

(Las comillas, son necesarias porque la carpeta "Tema 4" tiene un espacio).

2. **Crear un entorno virtual de Python:** Esto es para que las librerías no se peleen con las del sistema.

```
python3 -m venv venv
```

3. Activar el entorno virtual:

```
source venv/bin/activate
```

```
ubuntu@ip-172-31-90-78:~/SBD$ find . -iname "*aurora*" -o -iname "*fastapi*" -o
./Tema 1/Docker/requirements.txt
./Tema 4/webapp/backend/requirements.txt
./Tema 4/webapp/deploy/nginx/aurora.nginx.conf.template
./Tema 4/webapp/deploy/systemd/aurora-backend.service
./Tema 4/webapp/deploy/aurora.env.template
ubuntu@ip-172-31-90-78:~/SBD$ cd "/home/ubuntu/SBD/Tema 4/webapp/backend"
ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$ python3 -m venv venv
ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$ source venv/bin/activate
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$
```

(Veremos que ahora el prompt de la terminal empieza con `(venv)`, eso significa que estamos dentro).

4. Instalación de las dependencias del proyecto:

Aquí instalamos FastAPI, Uvicorn y todo lo que se encuentre en ese archivo que vimos en la búsqueda.

```
pip install -r requirements.txt
```

```
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$ pip install -r requirements.txt
Collecting fastapi>=0.110 (from -r requirements.txt (line 1))
  Downloading fastapi-0.135.2-py3-none-any.whl.metadata (28 kB)
Collecting uvicorn>=0.27 (from uvicorn[standard]>=0.27->-r requirements.txt (line 2))
  Downloading uvicorn-0.42.0-py3-none-any.whl.metadata (6.7 kB)
Collecting pydantic>=2.0 (from -r requirements.txt (line 3))
  Downloading pydantic-2.12.5-py3-none-any.whl.metadata (90 kB)
     ---------------------------------------- 90.6/90.6 kB 5.3 MB/s eta 0:00:00
Collecting python-dotenv>=1.0 (from -r requirements.txt (line 4))
  Downloading python_dotenv-1.2.2-py3-none-any.whl.metadata (27 kB)
Collecting starlette>=0.46.0 (from fastapi>=0.110->-r requirements.txt (line 1))
  Downloading starlette-1.0.0-py3-none-any.whl.metadata (6.3 kB)
Collecting typing-extensions>=4.8.0 (from fastapi>=0.110->-r requirements.txt (line 1))
  Downloading typing_extensions-4.15.0-py3-none-any.whl.metadata (3.3 kB)
Collecting typing-inspection>=0.4.2 (from fastapi>=0.110->-r requirements.txt (line 1))
  Downloading typing_inspection-0.4.2-py3-none-any.whl.metadata (2.6 kB)
Collecting annotated-doc>=0.0.2 (from fastapi>=0.110->-r requirements.txt (line 1))
  Downloading annotated_doc-0.0.4-py3-none-any.whl.metadata (6.6 kB)
Collecting click>=7.0 (from uvicorn>=0.27->uvicorn[standard]>=0.27->-r requirements.txt (line 2))
  Downloading click-8.3.1-py3-none-any.whl.metadata (2.6 kB)
Collecting h11>=0.8 (from uvicorn>=0.27->uvicorn[standard]>=0.27->-r requirements.txt (line 2))
  Downloading h11-0.16.0-py3-none-any.whl.metadata (8.3 kB)
Collecting annotated-types>=0.6.0 (from pydantic>=2.0->-r requirements.txt (line 3))
  Downloading annotated_types-0.7.0-py3-none-any.whl.metadata (15 kB)
Collecting pydantic-core==2.41.5 (from pydantic>=2.0->-r requirements.txt (line 3))
  Downloading pydantic_core-2.41.5-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (7.3 kB)
Collecting httptools>=0.6.3 (from uvicorn[standard]>=0.27->-r requirements.txt (line 2))
  Downloading httptools-0.7.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (3.5 kB)
Collecting pyyaml>=5.1 (from uvicorn[standard]>=0.27->-r requirements.txt (line 2))
  Downloading pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (2.4 kB)
Collecting uvloop>=0.15.1 (from uvicorn[standard]>=0.27->-r requirements.txt (line 2))
  Downloading uvloop-0.22.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl.metadata (4.9 kB)
Collecting watchfiles>=0.20 (from uvicorn[standard]>=0.27->-r requirements.txt (line 2))
  Downloading watchfiles-1.1.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl.metadata (4.9 kB)
Collecting websockets>=10.4 (from uvicorn[standard]>=0.27->-r requirements.txt (line 2))
  Downloading websockets-16.0-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl.metadata (6.8 kB)
Collecting anyio<5,>=3.6.2 (from starlette>=0.46.0->fastapi>=0.110->-r requirements.txt (line 1))
  Downloading anyio-4.13.0-py3-none-any.whl.metadata (4.5 kB)
Collecting idna>=2.8 (from anyio<5,>=3.6.2->starlette>=0.46.0->fastapi>=0.110->-r requirements.txt (line 1))
  Downloading idna-3.11-py3-none-any.whl.metadata (8.4 kB)
Downloading fastapi-0.135.2-py3-none-any.whl (117 kB)
```

```
Downloading python_dotenv-1.2.2-py3-none-any.whl (22 kB)
Downloading annotated_doc-0.0.4-py3-none-any.whl (5.3 kB)
Downloading annotated_types-0.7.0-py3-none-any.whl (13 kB)
Downloading click-8.3.1-py3-none-any.whl (108 kB)
   ---------------------------------------- 108.3/108.3 kB 10.4 MB/s eta 0:00:00
Downloading h11-0.16.0-py3-none-any.whl (37 kB)
Downloading httptools-0.7.1-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (517 kB)
   ---------------------------------------- 517.7/517.7 kB 26.2 MB/s eta 0:00:00
Downloading pyyaml-6.0.3-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (807 kB)
   ---------------------------------------- 807.9/807.9 kB 45.0 MB/s eta 0:00:00
Downloading starlette-1.0.0-py3-none-any.whl (72 kB)
   ---------------------------------------- 72.7/72.7 kB 6.6 MB/s eta 0:00:00
Downloading typing_extensions-4.15.0-py3-none-any.whl (44 kB)
   ---------------------------------------- 44.6/44.6 kB 3.9 MB/s eta 0:00:00
Downloading typing_inspection-0.4.2-py3-none-any.whl (14 kB)
Downloading uvloop-0.22.1-cp312-cp312-manylinux2014_x86_64.manylinux_2_17_x86_64.manylinux_2_28_x86_64.whl (4.4 MB)
   ---------------------------------------- 4.4/4.4 MB 116.2 MB/s eta 0:00:00
Downloading watchfiles-1.1.1-cp312-cp312-manylinux_2_17_x86_64.manylinux2014_x86_64.whl (456 kB)
   ---------------------------------------- 456.8/456.8 kB 41.4 MB/s eta 0:00:00
Downloading websockets-16.0-cp312-cp312-manylinux1_x86_64.manylinux_2_28_x86_64.manylinux_2_5_x86_64.whl (184 kB)
   ---------------------------------------- 184.9/184.9 kB 19.5 MB/s eta 0:00:00
Downloading anyio-4.13.0-py3-none-any.whl (114 kB)
   ---------------------------------------- 114.4/114.4 kB 12.2 MB/s eta 0:00:00
Downloading idna-3.11-py3-none-any.whl (71 kB)
   ---------------------------------------- 71.0/71.0 kB 7.2 MB/s eta 0:00:00
Installing collected packages: websockets, uvloop, typing-extensions, pyyaml, python-dotenv, idna, httptools, h11, click,
iles, starlette, pydantic, fastapi
Successfully installed annotated-doc-0.0.4 annotated-types-0.7.0 anyio-4.13.0 click-8.3.1 fastapi-0.135.2 h11-0.16.0 http
.3 starlette-1.0.0 typing-extensions-4.15.0 typing-inspection-0.4.2 uvicorn-0.42.0 uvloop-0.22.1 watchfiles-1.1.1 websock
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$
```

5. Ver qué archivos hay para arrancarlo:

```
ls -la
```

```
.3 starlette-1.0.0 typing-extensions-4.15.0 typing-inspection-0.4.
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$ ls -la
total 20
drwxr-xr-x 4 ubuntu ubuntu 4096 Mar 24 16:38 .
drwxr-xr-x 5 ubuntu ubuntu 4096 Mar 24 16:25 ..
drwxr-xr-x 2 ubuntu ubuntu 4096 Mar 24 16:25 app
-rw-r--r-- 1 ubuntu ubuntu   71 Mar 24 16:25 requirements.txt
drwxrwxr-x 5 ubuntu ubuntu 4096 Mar 24 16:38 venv
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$
```

La carpeta app es donde está el corazón de FastAPI.

Antes de arrancar el motor a ciegas, vamos a confirmar exactamente cómo se llama el archivo principal dentro de app y dónde se encuentra el código del "Front estático" que tiene que cargar la web .

Lanzamos estos dos comandos rápidos para echar un último vistazo al mapa:

```
ls -la app
ls -la ..
```

```
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$ ls -la app
ls -la ..
total 24
drwxr-xr-x 2 ubuntu ubuntu 4096 Mar 24 16:25 .
drwxr-xr-x 4 ubuntu ubuntu 4096 Mar 24 16:38 ..
-rw-r--r-- 1 ubuntu ubuntu 1973 Mar 24 16:25 logging_jsonl.py
-rw-r--r-- 1 ubuntu ubuntu 2217 Mar 24 16:25 main.py
-rw-r--r-- 1 ubuntu ubuntu 1925 Mar 24 16:25 middleware.py
-rw-r--r-- 1 ubuntu ubuntu 1235 Mar 24 16:25 models.py
total 20
drwxr-xr-x 5 ubuntu ubuntu 4096 Mar 24 16:25 .
drwxr-xr-x 6 ubuntu ubuntu 4096 Mar 24 16:25 ..
drwxr-xr-x 4 ubuntu ubuntu 4096 Mar 24 16:38 backend
drwxr-xr-x 4 ubuntu ubuntu 4096 Mar 24 16:25 deploy
drwxr-xr-x 4 ubuntu ubuntu 4096 Mar 24 16:25 frontend
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$
```

Tenemos el `main.py` del backend y, justo un nivel más arriba, la carpeta `frontend` con la web estática. Doy por hecho que se ha configurado el FastAPI para que sirva esa web directamente.

Arrancamos el motor. Necesitamos usar `sudo` (los puertos por debajo del 1024 requieren permisos de administrador en Linux).

Nos aseguramos de estar en la carpeta `backend` y lanzamos este comando exacto.

```
sudo ./venv/bin/python -m uvicorn app.main:app --host
0.0.0.0 --port 80
```

```
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$ sudo ./venv/bin/python -m uvicorn app.main:app --host 0.0.0.0 --port 80
INFO:     Started server process [3246]
INFO:     Waiting for application startup.
INFO:     Application startup complete.
INFO:     Uvicorn running on http://0.0.0.0:80 (Press CTRL+C to quit)
```

Apreciamos que:

1. En la terminal negra salen unas letras diciendo *«Uvicorn running on http://0.0.0.0:80»*.
2. Vamos a la consola de AWS, copiamos la **IP Pública** de esta máquina antes (`EC2-6_Web_CloudWatch`).
3. Pegamos esa IP directamente en tu navegador (sin poner `:8080` ni nada, solo la IP).

Detectamos un problema porque arrancamos FastAPI en el puerto 80, como dice la documentación de la práctica, FastAPI es solo el *backend* que tiene el endpoint `POST /track` para recibir los eventos. Es decir, no tiene el código HTML visual de la página web.

Vamos a hacerlo bien usando las plantillas Nginx quién será el encargado de mostrar la web por el puerto 80 y mandarle los eventos al backend.

Segue estos pasos en esa misma terminal:

Vamos a leer la plantilla para entender cómo conectamos el front y el back, lanzando este comando:

```
cat "/home/ubuntu/SBD/Tema 4/webapp/deploy/nginx/aurora.nginx.conf.template"
```

```
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$ cat "/home/ubuntu/SBD/Tema 4/webapp/deploy/nginx/aurora.nginx.conf.template"
# webapp/deploy/nginx/aurora.nginx.conf.template
server {
    listen 80 default_server;
    listen [::]:80 default_server;

    server_name _;

    # Front estático
    root /var/www/aurora/frontend;
    index index.html;

    # Logs Nginx (útiles para depurar)
    access_log /var/log/nginx/aurora_access.log;
    error_log  /var/log/nginx/aurora_error.log warn;

    # Seguridad mínima (sin complicar)
    add_header X-Content-Type-Options nosniff always;

    # --- PROXY A FASTAPI ---
    # Track endpoint (client-side)
    location = /track {
        proxy_pass http://127.0.0.1:8000/track;

        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Para POST con JSON
        proxy_read_timeout 30s;
        proxy_connect_timeout 5s;
    }

    # Health endpoint (para comprobación)
    location = /health {
        proxy_pass http://127.0.0.1:8000/health;
```

```
    # --- PROXY A FASTAPI ---
    # Track endpoint (client-side)
    location = /track {
        proxy_pass http://127.0.0.1:8000/track;

        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;

        # Para POST con JSON
        proxy_read_timeout 30s;
        proxy_connect_timeout 5s;
    }

    # Health endpoint (para comprobación)
    location = /health {
        proxy_pass http://127.0.0.1:8000/health;

        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }

    # --- SERVIR FRONT ---
    # Archivos estáticos típicos (mejor cache)
    location ~* \.(css|js|png|jpg|jpeg|gif|svg|ico)$ {
        try_files $uri =404;
        expires 1h;
        add_header Cache-Control "public, max-age=3600";
    }

    # Datos (events.json)
    location /data/ {
        try_files $uri =404;
        expires 5m;
        add_header Cache-Control "public, max-age=300";
    }

    # HTML: sin cache agresiva (para ver cambios)
    location / {
        try_files $uri $uri/ =404;
        add_header Cache-Control "no-store";
    }
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$
```

Vemos que Nginx está diseñado para que escuche en el puerto 80 (la web normal). Nginx servirá los archivos HTML/CSS desde `/var/www/aurora/frontend`, y cuando reciba datos en `/track`, los mandará a nuestro FastAPI que estará corriendo escondido en el puerto 8000.

Vamos a montar esto paso a paso:

Instalar Nginx y copiar el Frontend:

Ejecutamos estos comandos uno a uno, el objetivo es instalar el servidor, crear la carpeta que se pide y copiar los archivos de la web ahí:

```
sudo apt-get install -y nginx

sudo mkdir -p /var/www/aurora

sudo cp -r "/home/ubuntu/SBD/Tema 4/webapp/frontend"
/var/www/aurora/
```

```
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes... [==================================================
Scanning processes...

Scanning linux images... [
Scanning linux images... [==================================================
Scanning linux images... [==================================================
Scanning linux images... [==================================================
Scanning linux images...


Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$
```

Configurar Nginx:

Vamos a copiar la plantilla donde Nginx la pueda leer y vamos a borrar la que viene por defecto para que no estorbe:

```
sudo cp "/home/ubuntu/SBD/Tema
4/webapp/deploy/nginx/aurora.nginx.conf.template"
/etc/nginx/sites-available/aurora

sudo ln -s /etc/nginx/sites-available/aurora
/etc/nginx/sites-enabled/

sudo rm /etc/nginx/sites-enabled/default

sudo systemctl restart nginx
```

Arrancar el Backend (FastAPI):

Vamos a arrancar el cerebro en el puerto 8000, tal y como lo espera Nginx. Como lo queremos dejar corriendo en segundo plano, usamos nohup:

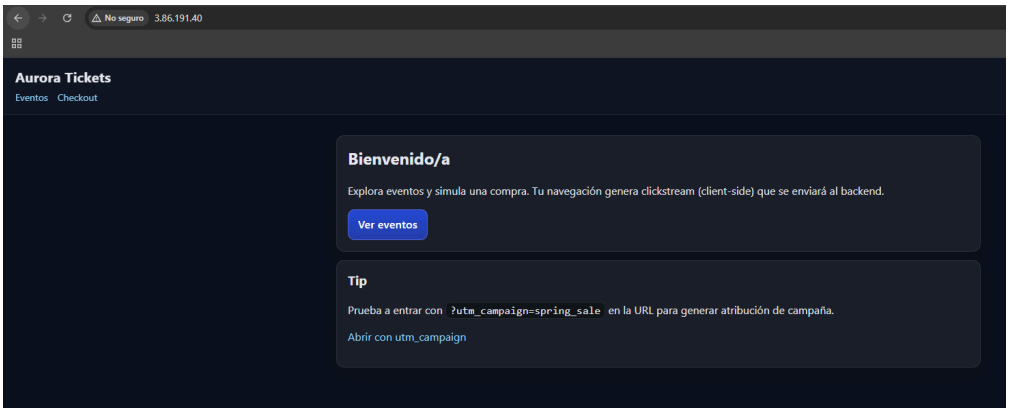
```
cd "/home/ubuntu/SBD/Tema 4/webapp/backend"

nohup ./venv/bin/python -m uvicorn app.main:app --host
127.0.0.1 --port 8000 > backend.log 2>&1 &
```

(Al lanzar este último comando parece que no hace nada pero está corriendo en segundo plano).

Ahora sí volvemos al navegador y recargando la pestaña con la **IP Pública** de la máquina (`3.86.191.40`) incluyendo el http: `http://3.86.191.40`.

La práctica nos exige generar **al menos 200.000 eventos de clickstream** que simulen 7 días de tráfico, mezclando clicks a mano y una generación automática masiva .

Aurora Tickets

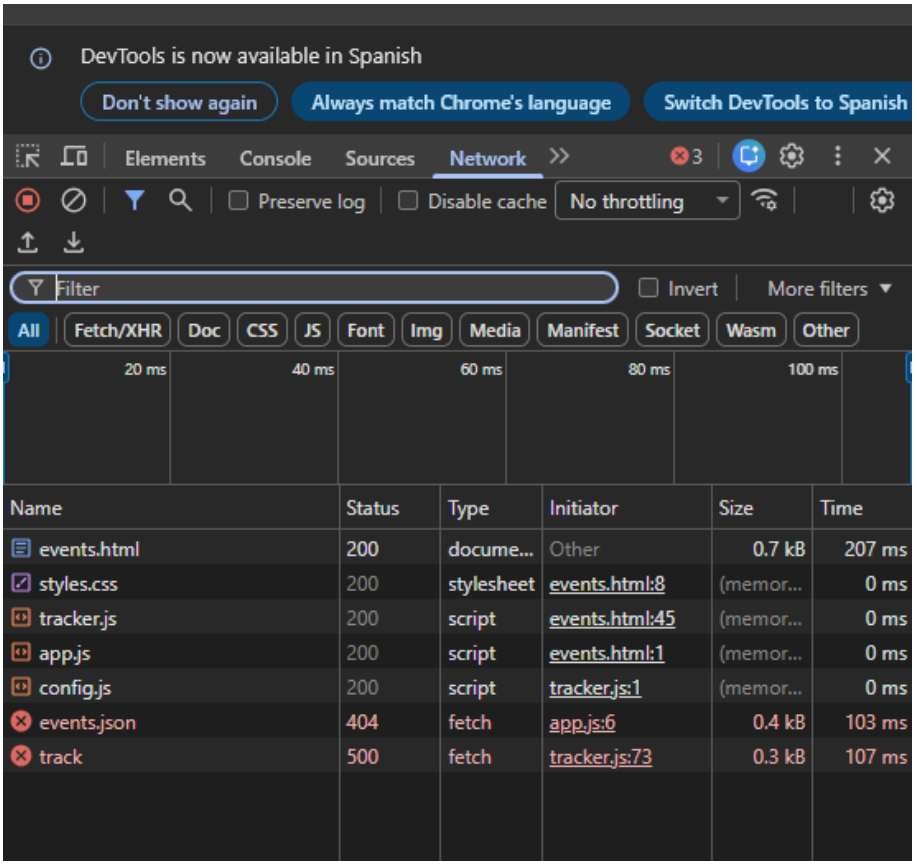


Vamos a comprobar que el registro de eventos está funcionando antes de soltar a la bestia (el script masivo):

Entrando en eventos apreciamos que no muestra eventos

Paso 1: Mira en el navegador (F12):

1. En esa misma pestaña de "Eventos", pulsamos la tecla **F12** del teclado (o hacemos clic derecho e "Inspeccionar").
2. Ir a la pestaña **Consola** (Console) o **Red** (Network).
3. Recarga la página (F5).
4. Veremos que sale algo en rojo (seguramente un error 404 o 500).



Error 404 (`events.json`): La web está buscando el catálogo de eventos para mostrarlo en pantalla y no lo encuentra. Seguramente hay que generarlo o copiarlo.

Error 500 (`/track`): El backend de Python (FastAPI) ha intentado guardar el registro del clic y ha reventado por dentro. Seguramente por algún tema de permisos de escritura o falta de un archivo.

Vamos a cazar los dos fallos, abriendo la terminal negra de la máquina `EC2-6` y lanzamos estos dos comandos para ver qué está pasando exactamente:

1. Para ver por qué peta el backend (el Error 500):

```
tail -n 15 "/home/ubuntu/SBD/Tema
4/webapp/backend/backend.log"
```

2. Para buscar dónde está el generador de datos (para el Error 404): Vamos a ver qué carpetas hay alrededor:

```
ls -la "/home/ubuntu/SBD/Tema 4"

ls -la "/home/ubuntu/SBD/Tema 4/webapp"
```

```
Mar 24 17:08:15 ip-172-31-90-78 systemd[1]: Starting nginx.service - A high performance web server and a reverse proxy server...
Mar 24 17:08:15 ip-172-31-90-78 systemd[1]: Started nginx.service - A high performance web server and a reverse proxy server.
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$ tail -n 15 "/home/ubuntu/SBD/Tema 4/webapp/backend/backend.log"
  File "/home/ubuntu/SBD/Tema 4/webapp/backend/venv/lib/python3.12/site-packages/starlette/middleware/base.py", line 191, in __call__
    with recv_stream, send_stream, collapse_excgroups():
  File "/usr/lib/python3.12/contextlib.py", line 158, in __exit__
    self.gen.throw(value)
  File "/home/ubuntu/SBD/Tema 4/webapp/backend/venv/lib/python3.12/site-packages/starlette/_utils.py", line 87, in collapse_excgroups
    raise exc
  File "/home/ubuntu/SBD/Tema 4/webapp/backend/venv/lib/python3.12/site-packages/starlette/middleware/base.py", line 193, in __call__
    response = await self.dispatch_func(request, call_next)
               ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ubuntu/SBD/Tema 4/webapp/backend/app/middleware.py", line 57, in dispatch
    self.writer.append(event)
  File "/home/ubuntu/SBD/Tema 4/webapp/backend/app/logging_jsonl.py", line 51, in append
    with open(self.path, "a", encoding="utf-8") as f:
         ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
PermissionError: [Errno 13] Permission denied: '/var/log/aurora/aurora_clickstream.jsonl'
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$ ls -la "/home/ubuntu/SBD/Tema 4"
ls -la "/home/ubuntu/SBD/Tema 4/webapp"
total 32
drwxr-xr-x 6 ubuntu ubuntu 4096 Mar 24 16:25 .
drwxr-xr-x 7 ubuntu ubuntu 4096 Mar 24 16:25 ..
-rw-r--r-- 1 ubuntu ubuntu 6191 Mar 24 16:25 README.md
drwxr-xr-x 2 ubuntu ubuntu 4096 Mar 24 16:25 cloudwatch
drwxr-xr-x 2 ubuntu ubuntu 4096 Mar 24 16:25 generators
drwxr-xr-x 2 ubuntu ubuntu 4096 Mar 24 16:25 simulators
drwxr-xr-x 5 ubuntu ubuntu 4096 Mar 24 16:25 webapp
total 20
drwxr-xr-x 5 ubuntu ubuntu 4096 Mar 24 16:25 .
drwxr-xr-x 6 ubuntu ubuntu 4096 Mar 24 16:25 ..
drwxr-xr-x 4 ubuntu ubuntu 4096 Mar 24 17:08 backend
drwxr-xr-x 4 ubuntu ubuntu 4096 Mar 24 16:25 deploy
drwxr-xr-x 4 ubuntu ubuntu 4096 Mar 24 16:25 frontend
(venv) ubuntu@ip-172-31-90-78:~/SBD/Tema 4/webapp/backend$
```

El backend de FastAPI está intentando escribir el clic en el archivo /var/log/aurora/aurora_clickstream.jsonl, pero Linux le está diciendo que no puede, que no tiene permisos para escribir ahí. Por eso revienta y da el Error 500.

Además, en el resultado del ls apreciamos dos carpetas clave: generators y simulators.

- En generators está el script que crea los datos de negocio (el catálogo para arreglar el Error 404 y los CSV con errores que pide el PDF).
- En simulators esta la bestia que va a generar los 200.000 eventos automáticos .

Vamos a arreglar los permisos y a cotillear esas carpetas todo eso lanzando esto en la terminal negra:

Arreglar el Error 500 (Permisos del log):

Vamos a crear el archivo a mano y a darle permisos absolutos para que FastAPI pueda escribir sin que Linux le corte el rollo:

```
sudo touch /var/log/aurora/aurora_clickstream.jsonl

sudo chown -R ubuntu:ubuntu /var/log/aurora

sudo chmod -R 777 /var/log/aurora
```

Vamos a generar `events.json` para que la web deje de quejarse y cargue el catálogo.

1. Vamos a la carpeta de los generadores de datos:

   ```
   cd "/home/ubuntu/SBD/Tema 4/generators"
   ```

2. Lista los archivos que hay dentro para ver cómo se llama el script de Python:

   ```
   ls -la
   ```

   Vemos un archivo llamado `generate_business_data.py` lo ejecutamos con Python:

   ```
   python3 generate_business_data.py
   ```

   Apreciamos un script para que cada alumno genere un conjunto de datos único basado en su ID.

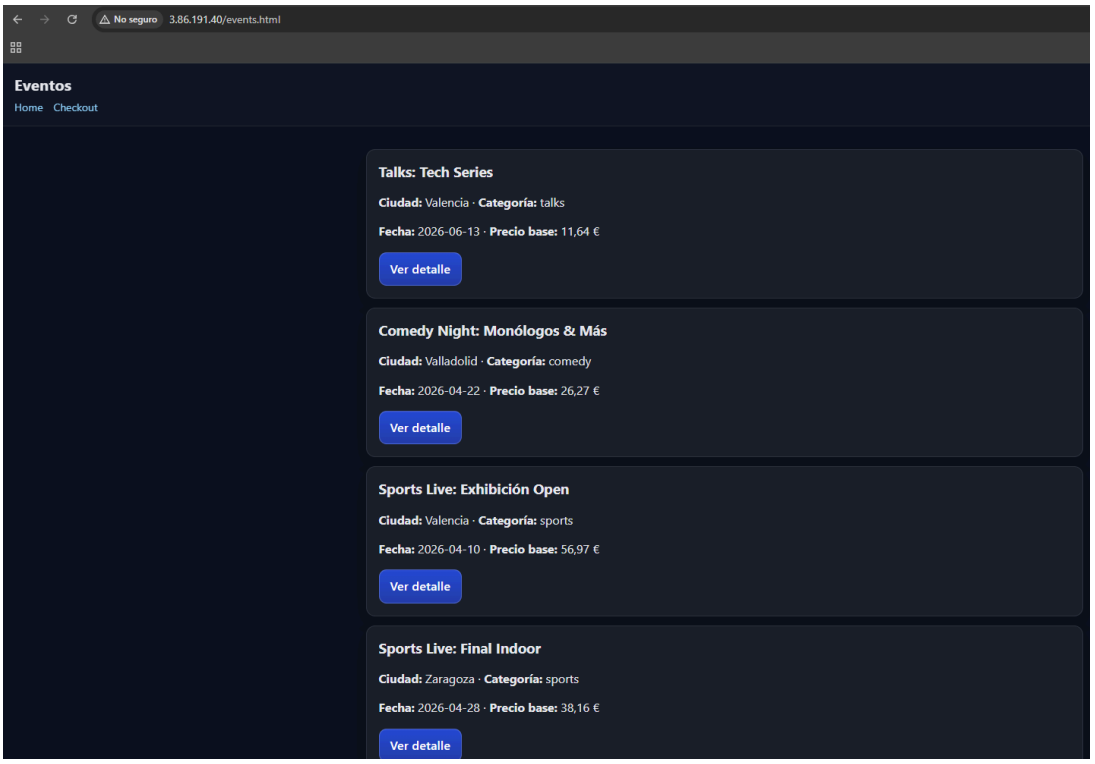
   Para que funcione, solo volvemos a lanzar el comando añadiendo ese parámetro con un identificador.

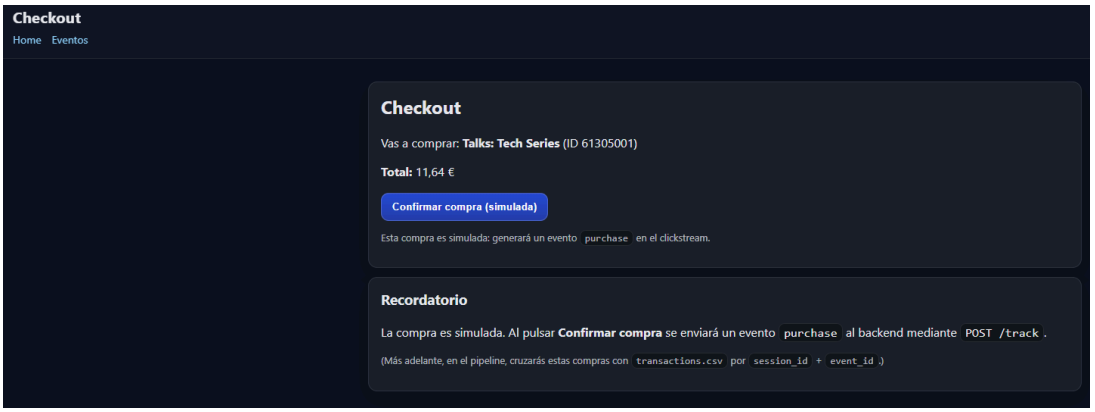
   Ejecutamos esto poniendo `12345678` como identificador:

   ```
   python3 generate_business_data.py --student-id
   12345678
   ```

```
generate_business_data.py: error: the following arguments are required: --student-id
ubuntu@ip-172-31-90-78:~/SBD/Tema 4/generators$ python3 generate_business_data.py --student-id 12345678
✅Generación completada
 - student_id: 12345678
 - seed: 4017716353
 - days: 7 (start: 2026-03-18)
 - events.csv: ./output_business/events.csv (total=120)
 - campaigns.csv: ./output_business/campaigns.csv (total=12)
 - transactions.csv: ./output_business/transactions.csv (total=20000)
 - events.json (web): ./output_business/events.json (active_valid=108)
ℹ️Nota: events.json contiene SOLO eventos activos/válidos (subconjunto consistente con events.csv).
ubuntu@ip-172-31-90-78:~/SBD/Tema 4/generators$
```

Ahora solo tenemos que coger ese archivo `events.json` y ponerlo donde Nginx lo está buscando (en la carpeta `data` del frontend).





Simular tráfico masivo:

Vamos a buscar el script que se ha preparado para simular a miles de usuarios navegando por la web.

En la terminal de la máquina EC2-6, ir a la carpeta de los simuladores:

```
cd "/home/ubuntu/SBD/Tema 4/simulators"
```

Listamos los archivos para ver qué tenemos:

```
ls -la
```

```
ubuntu@ip-172-31-90-78:~/SBD/Tema 4/simulators$ ls -la
total 36
drwxr-xr-x 2 ubuntu ubuntu  4096 Mar 24 16:25 .
drwxr-xr-x 6 ubuntu ubuntu  4096 Mar 24 16:25 ..
-rw-r--r-- 1 ubuntu ubuntu  3016 Mar 24 16:25 README_simulation.md
-rw-r--r-- 1 ubuntu ubuntu 14504 Mar 24 16:25 replay_clickstream.py
-rw-r--r-- 1 ubuntu ubuntu  6208 Mar 24 16:25 traffic_driver.py
ubuntu@ip-172-31-90-78:~/SBD/Tema 4/simulators$
```

Lanzamos este comando en la terminal para ver el manual de instrucciones del `traffic_driver.py`:

Vamos a decirle que simule **30.000 sesiones** (usuarios) y que cada uno haga hasta **15 acciones** por la web. Además, le vamos a poner `--sleep-ms 0` para que no haya pausas entre clics y termine lo antes posible.

Copiamos y pegamos este súper comando exacto en la terminal (he puesto las rutas absolutas para que no falle buscando los archivos):

```
python3 traffic_driver.py --base-url http://127.0.0.1
--student-id 12345678 --events-json "/home/ubuntu/SBD/Tema
4/generators/output_business/events.json" --campaigns-csv
"/home/ubuntu/SBD/Tema
4/generators/output_business/campaigns.csv" --sessions 30000
--max-actions 15 --sleep-ms 0
```

```
ubuntu@ip-172-31-90-78:~/SBD/Tema 4/simulators$ python3 traffic_driver.py --base-url http://127.0.0.1 --student-id 12345678 --
igns-csv "/home/ubuntu/SBD/Tema 4/generators/output_business/campaigns.csv" --sessions 30000 --max-actions 15 --sleep-ms 0
✅Traffic driver terminado
 - base_url: http://127.0.0.1
 - student_id: 12345678
 - seed: 4017716353
 - sessions: 30000
 - total_posts: 292705
 - ok_posts: 292705
ℹ Cada POST /track suele generar 2 líneas en JSONL (server_request + client event).
ubuntu@ip-172-31-90-78:~/SBD/Tema 4/simulators$
```

Guardar la evidencia obligatoria:

Lanzar este comando en la terminal para contar las líneas exactas del fichero. **Hacemos una captura de pantalla al resultado junto con el comando**, porque es una de las evidencias obligatorias para demostrar el volumen de datos generado:

```
wc -l /var/log/aurora/aurora_clickstream.jsonl
```

```
Last login: Tue Mar 24 18:40:46 2026 from 18.206.107.28
ubuntu@ip-172-31-90-78:~$ wc -l /var/log/aurora/aurora_clickstream.jsonl
585455 /var/log/aurora/aurora_clickstream.jsonl
ubuntu@ip-172-31-90-78:~$
```

5. Salto a CloudWatch

Preparamos el salto:

Ahora tenemos que decirle al agente de Amazon que lea ese archivo gigante y lo envíe a AWS. Antes vimos de refilón que había una carpeta para esto.

Vamos a ver qué contiene:

```
ls -la "/home/ubuntu/SBD/Tema 4/cloudwatch"
```

```
Last login: Tue Mar 24 18:40:46 2026 from 18.206.107.28
ubuntu@ip-172-31-90-78:~$ wc -l /var/log/aurora/aurora_clickstream.jsonl
585455 /var/log/aurora/aurora_clickstream.jsonl
ubuntu@ip-172-31-90-78:~$ ls -la "/home/ubuntu/SBD/Tema 4/cloudwatch"
total 16
drwxr-xr-x 2 ubuntu ubuntu 4096 Mar 24 16:25 .
drwxr-xr-x 6 ubuntu ubuntu 4096 Mar 24 16:25 ..
-rw-r--r-- 1 ubuntu ubuntu 3104 Mar 24 16:25 README_cloudwatch.md
-rw-r--r-- 1 ubuntu ubuntu  609 Mar 24 16:25 cw_agent_config.template.json
ubuntu@ip-172-31-90-78:~$
```

Queremos agarrar esa bestialidad de archivo y decirle a AWS que lo suba a la nube usando **CloudWatch**.

```
cat "/home/ubuntu/SBD/Tema
4/cloudwatch/cw_agent_config.template.json"
```

```
Last login: Tue Mar 24 18:40:46 2026 from 18.206.107.28
ubuntu@ip-172-31-90-78:~$ wc -l /var/log/aurora/aurora_clickstream.jsonl
585455 /var/log/aurora/aurora_clickstream.jsonl
ubuntu@ip-172-31-90-78:~$ ls -la "/home/ubuntu/SBD/Tema 4/cloudwatch"
total 16
drwxr-xr-x 2 ubuntu ubuntu 4096 Mar 24 16:25 .
drwxr-xr-x 6 ubuntu ubuntu 4096 Mar 24 16:25 ..
-rw-r--r-- 1 ubuntu ubuntu 3104 Mar 24 16:25 README_cloudwatch.md
-rw-r--r-- 1 ubuntu ubuntu  609 Mar 24 16:25 cw_agent_config.template.json
ubuntu@ip-172-31-90-78:~$ cat "/home/ubuntu/SBD/Tema 4/cloudwatch/cw_agent_config.template.json"
{
  "agent": {
    "metrics_collection_interval": 60,
    "logfile": "/opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log",
    "run_as_user": "root"
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/aurora/aurora_clickstream.jsonl",
            "log_group_name": "/aurora/PON_TU_NOMBRE_CW_AGENT_CONFIG_TEMPLATE/clickstream",
            "log_stream_name": "{instance_id}",
            "timezone": "UTC",
            "retention_in_days": 14
          }
        ]
      }
    },
    "force_flush_interval": 5
  }
}
```

Para que la máquina EC2-6 pueda enviar esos logs a CloudWatch, **necesita un "pase VIP" (un Rol IAM)**. Si no se lo ponemos, el agente lo intentará subir pero AWS le cerrará la puerta en las narices.

Vamos a la web de AWS y hacemos esto:

1. Ir a tus instancias y seleccionamos la **EC2-6_Web_CloudWatch**.
2. Arriba a la derecha, le damos a **Acciones** > **Seguridad** > **Modificar rol de IAM**.
3. En el desplegable, buscamos y seleccionamos el rol que tenga permisos de CloudWatch (LabRole).
4. Darle a **Actualizar rol de IAM**.

Copiar y editar la plantilla:

Primero copiamos la plantilla para no machacar la original y la abrimos con `nano`:

```
cd "/home/ubuntu/SBD/Tema 4/cloudwatch"
cp cw_agent_config.template.json cw_agent_config.json
nano cw_agent_config.json
```

Dentro del editor `nano`, nos movemos con las flechas hasta la línea de `"log_group_name"` y pongo mi nombre sin espacios. (Ejemplo: `/aurora/Inigo/clickstream`).

Pulsar `Ctrl + O`, luego `Enter` para guardar, y `Ctrl + X` para salir.

```
{
  "agent": {
    "metrics_collection_interval": 60,
    "logfile": "/opt/aws/amazon-cloudwatch-agent/logs/amazon-cloudwatch-agent.log",
    "run_as_user": "root"
  },
  "logs": {
    "logs_collected": {
      "files": {
        "collect_list": [
          {
            "file_path": "/var/log/aurora/aurora_clickstream.jsonl",
            "log_group_name": "/aurora/Inigo/clickstream",
            "log_stream_name": "{instance_id}",
            "timezone": "UTC",
            "retention_in_days": 14
          }
        ]
      }
    },
    "force_flush_interval": 5
  }
}
```

Arrancar el Agente de CloudWatch:

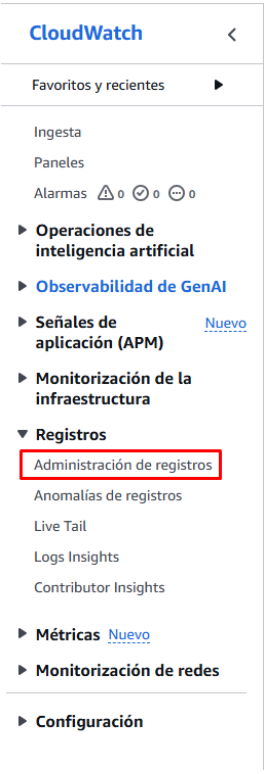
Ahora que tenemos la configuración lista y el Rol IAM asignado, vamos a enchufarlo a AWS con este comando que le pasa el archivo de configuración al agente:

```
sudo
/opt/aws/amazon-cloudwatch-agent/bin/amazon-cloudwatch-agent
-ctl -a fetch-config -m ec2 -s -c
file:/home/ubuntu/SBD/Tema\
4/cloudwatch/cw_agent_config.json
```

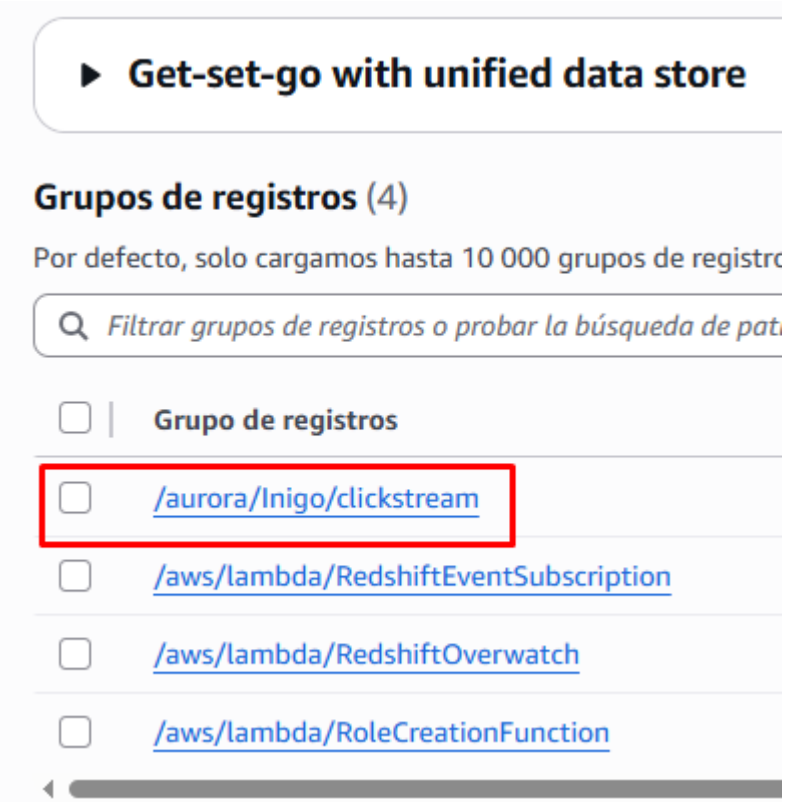
Si todo va bien, nos dirá que ha arrancado (Status: running).

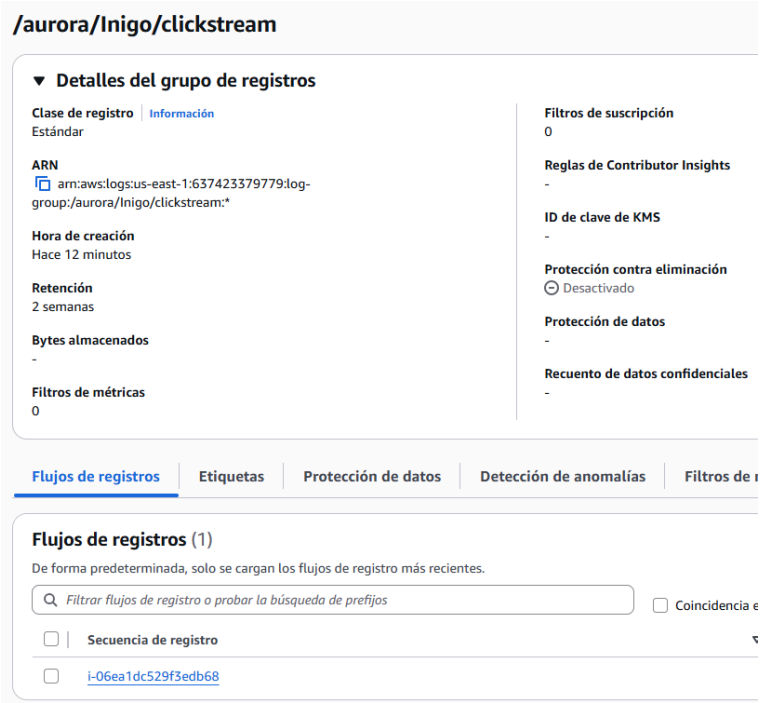
Para comprobar que los datos están subiendo, vamos a la consola de AWS:

1. Buscamos **CloudWatch** en la barra de arriba.
2. En el menú de la izquierda, vamos a **Grupos de registros** (Log groups).
3. Debería aparecer el grupo /aurora/Inigo/clickstream y dentro deberían estar empezando a llegar los cientos de miles de clics.



Aurora Tickets





6. Creación de Bucket y Jobs.

- **Problema:** Aurora Tickets tiene mucho tráfico web pero pocas compras (baja conversión). La web va lenta a veces, da errores y hay sospechas de bots. Falta analítica Big Data .
- **Objetivos del Proyecto:** Tenemos que crear una solución en AWS, capturar el comportamiento de los usuarios (clickstream), monitorizar en tiempo real con CloudWatch y procesar datos masivos con Apache Spark .
- **KPIs (Métricas clave):** Tasa de conversión (del clic a la compra), ratio de interés vs ingresos, cantidad de errores/latencia y volumen de tráfico anómalo.
- **Decisiones de Negocio:** Con esta información, la empresa decidirá qué campañas de marketing mantener, cómo bloquear bots y qué eventos promocionar más.

Preparación de datos:

Para empezar esta fase, necesitamos nuestro Data Lake.

Debemos crear un "cubo" (Bucket) en Amazon S3 y crear dentro la estructura obligatoria: la carpeta `raw/` . Ahí subiremos los archivos CSV (catálogo, campañas y transacciones) que generamos antes .

Se nos exige organizar los datos en Amazon S3 usando tres capas obligatorias: `raw/` (aterrizaje sin transformar), `curated/` (limpio en Parquet) y `analytics/` (datasets finales). Además, la ruta tiene que incluir mi nombre.

Como los buckets en S3 tienen que tener un nombre único en todo el mundo, vamos a crearlo directamente desde la terminal de EC2-6, que es lo más rápido.

Copiamos y pegamos estos comandos uno a uno en la terminal negra. He puesto `aurora-tickets-inigo` como nombre del cubo.

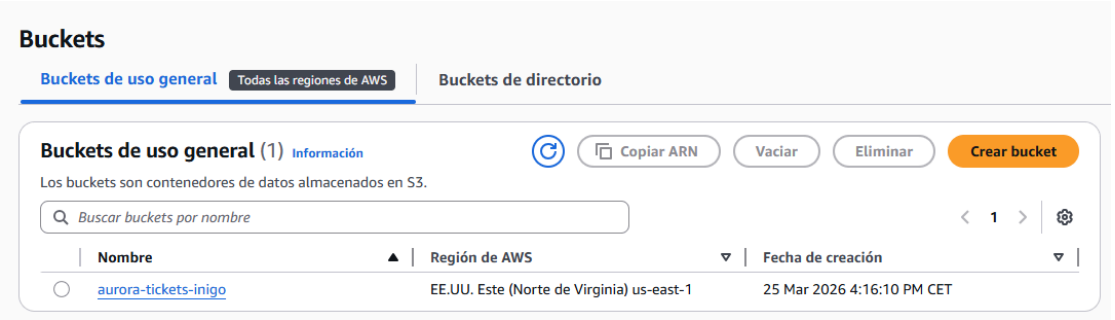
```
# 1. Definimos el nombre del cubo (cámbialo si te da error
de que ya existe)

BUCKET="aurora-tickets-inigo"



# 2. Creamos el bucket en S3

aws s3 mb s3://$BUCKET
```

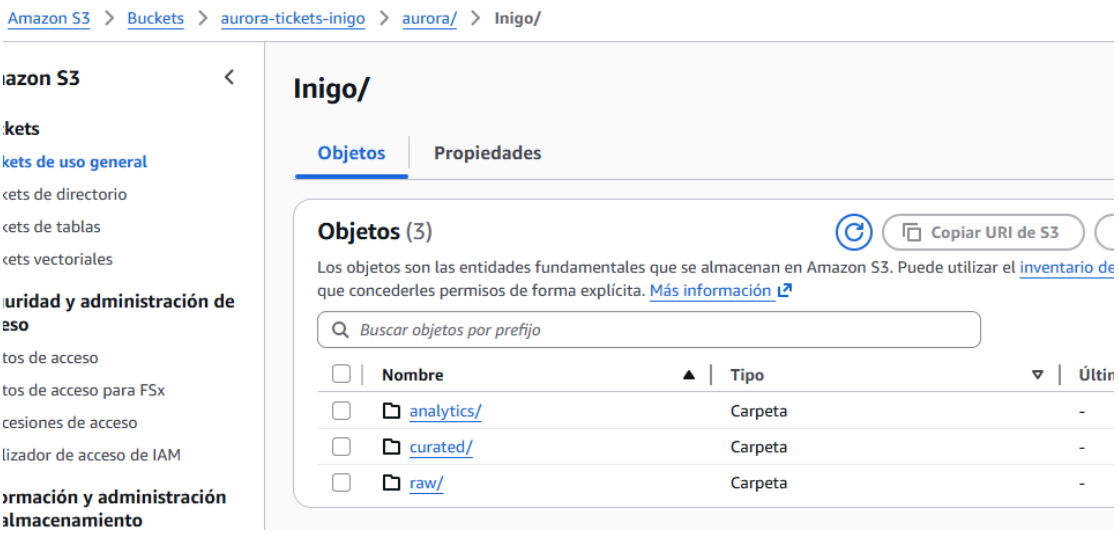
```
# 3. Subimos los 3 archivos CSV de negocio a la capa "raw"

aws s3 cp "/home/ubuntu/SBD/Tema
4/generators/output_business/events.csv"
s3://$BUCKET/aurora/Inigo/raw/events.csv

aws s3 cp "/home/ubuntu/SBD/Tema
4/generators/output_business/campaigns.csv"
s3://$BUCKET/aurora/Inigo/raw/campaigns.csv

aws s3 cp "/home/ubuntu/SBD/Tema
4/generators/output_business/transactions.csv"
s3://$BUCKET/aurora/Inigo/raw/transactions.csv


# 4. Creamos las carpetas vacías de curated y analytics para
tener la estructura lista

aws s3api put-object --bucket $BUCKET --key
aurora/Inigo/curated/

aws s3api put-object --bucket $BUCKET --key
aurora/Inigo/analytics/
```

```
Last login: Wed Mar 25 10:39:25 2026 from 18.206.107.28
ubuntu@ip-172-31-90-78:~$ aws --version
aws-cli/2.34.16 Python/3.14.3 Linux/6.17.0-1009-aws exe/x86_64.ubuntu.24
ubuntu@ip-172-31-90-78:~$ BUCKET="aurora-tickets-inigo"
ubuntu@ip-172-31-90-78:~$ aws s3 mb s3://$BUCKET
make_bucket: aurora-tickets-inigo
ubuntu@ip-172-31-90-78:~$ aws s3 cp "/home/ubuntu/SBD/Tema 4/generators/output_business/events.csv" s3://$BUCKET/aurora/Inigo/raw/events.csv
aws s3 cp "/home/ubuntu/SBD/Tema 4/generators/output_business/campaigns.csv" s3://$BUCKET/aurora/Inigo/raw/campaigns.csv
aws s3 cp "/home/ubuntu/SBD/Tema 4/generators/output_business/transactions.csv" s3://$BUCKET/aurora/Inigo/raw/transactions.csv
upload: SBD/Tema 4/generators/output_business/events.csv to s3://aurora-tickets-inigo/aurora/Inigo/raw/events.csv
upload: SBD/Tema 4/generators/output_business/campaigns.csv to s3://aurora-tickets-inigo/aurora/Inigo/raw/campaigns.csv
upload: SBD/Tema 4/generators/output_business/transactions.csv to s3://aurora-tickets-inigo/aurora/Inigo/raw/transactions.csv
ubuntu@ip-172-31-90-78:~$ aws s3api put-object --bucket $BUCKET --key aurora/Inigo/curated/
aws s3api put-object --bucket $BUCKET --key aurora/Inigo/analytics/
{
    "ETag": "\"d41d8cd98f00b204e9800998ecf8427e\"",
    "ChecksumCRC64NVME": "AAAAAAAAAAA=",
    "ChecksumType": "FULL_OBJECT",
    "ServerSideEncryption": "AES256"
}
{
    "ETag": "\"d41d8cd98f00b204e9800998ecf8427e\"",
    "ChecksumCRC64NVME": "AAAAAAAAAAA=",
    "ChecksumType": "FULL_OBJECT",
    "ServerSideEncryption": "AES256"
}
ubuntu@ip-172-31-90-78:~$
```

Hemos creado el bucket `aurora-tickets-inigo` y hemos subido los tres archivos CSV a la capa `raw/` a la primera. Además, el último comando nos confirma que las carpetas vacías `curated/` y `analytics/` ya están inicializadas.

Aurora Tickets





Job 1 (Curación):

Para crear el `job1_curation.py`, necesitamos escribir un script de PySpark que haga exactamente:

1. **Leer de S3 raw/:** Cargar los 3 CSV que subimos antes y los logs de clickstream que están en CloudWatch.
2. **Limpiar:** Quitar nulos, arreglar fechas fuera de rango y quitar IDs huérfanos.
3. **Escribir a S3 curated/:** Guardarlo todo en formato Parquet, particionado obligatoriamente por la columna `dt` (fecha) y otra dimensión adicional (ej: `event_type`) .

Este código lee los datos crudos, elimina nulos, filtra importes negativos y cruza tablas para eliminar registros huérfanos. Finalmente, guarda todo en formato Parquet particionado.

En la máquina **EC2-5_Submit**, ejecutamos `nano job1_curation.py` y pegamos este código exacto:

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import col, to_date


spark = SparkSession.builder \

    .appName("Job1_Curation") \

    .getOrCreate()


ruta_base = "s3a://aurora-tickets-inigo/aurora/Inigo"


df_events = spark.read.option("header",
"true").csv(f"{ruta_base}/raw/events.csv")

df_campaigns = spark.read.option("header",
"true").csv(f"{ruta_base}/raw/campaigns.csv")

df_transactions = spark.read.option("header",
"true").csv(f"{ruta_base}/raw/transactions.csv")

df_clickstream =
spark.read.json(f"{ruta_base}/raw/aurora_clickstream.jsonl")


df_events_clean = df_events.dropna(subset=["event_id"])


df_campaigns_clean =
df_campaigns.dropna(subset=["utm_campaign"])
```

```
df_transactions_clean =
df_transactions.dropna(subset=["transaction_id",
"session_id", "event_id"]) \

    .filter(col("amount") > 0) \

    .join(df_events_clean, "event_id", "left_semi")



df_clickstream_clean =
df_clickstream.dropna(subset=["timestamp", "session_id"]) \

    .withColumn("dt", to_date(col("timestamp"))) \

    .filter(col("dt").isNotNull())



df_events_clean.write.mode("overwrite").parquet(f"{ruta_base
}/curated/events")



df_campaigns_clean.write.mode("overwrite").parquet(f"{ruta_b
ase}/curated/campaigns")



df_transactions_clean.withColumn("dt",
to_date(col("transaction_date"))) \


.write.mode("overwrite").partitionBy("dt").parquet(f"{ruta_b
ase}/curated/transactions")



df_clickstream_clean.write.mode("overwrite").partitionBy("dt
", "event_type").parquet(f"{ruta_base}/curated/clickstream")



spark.stop()
```

Guardar el archivo (`Ctrl + O`, `Enter`, `Ctrl + X`).

Paso previo obligatorio:

El script busca el archivo `aurora_clickstream.jsonl` en la carpeta `raw/` de tu S3 pero actualmente ese archivo gigante de casi 600.000 líneas está en la máquina `EC2-6`.

Tenemos que ir a la terminal de la **`EC2-6`** y subirlo a S3 con este comando:

```
aws s3 cp /var/log/aurora/aurora_clickstream.jsonl
s3://aurora-tickets-inigo/aurora/Inigo/raw/aurora_clickstrea
m.jsonl
```

Ejecutar el Job:

Una vez subido el log, volvemos a la terminal de **`EC2-5_Submit`** y lanzamos el trabajo al clúster indicando la IP de Máster:

```
park-submit --master spark://LA_IP_PRIVADA_DEL_MASTER:7077
job1_curation.py
```

LA_IP_PRIVADA_DEL_MASTER = 172.31.14.249

Por lo tanto:

```
spark-submit --master spark://172.31.14.249:7077
job1_curation.py
```

Apache Spark (que estamos usando en la versión 3.5.* como pide la práctica) viene "limpio" por defecto y no trae las librerías necesarias para hablar con los buckets de Amazon S3 usando el protocolo `s3a://`. Le falta el "diccionario" para entenderse con AWS.

Para solucionarlo, solo tenemos que decirle a spark-submit que se descargue ese diccionario de internet al vuelo antes de empezar a trabajar. Lo hacemos usando el parámetro --packages.

Lanzamos este comando actualizado en la terminal **EC2-5_Submit**:

```
spark-submit --packages org.apache.hadoop:hadoop-aws:3.3.4
--master spark://172.31.14.249:7077 job1_curation.py
```

(Al lanzarlo la primera vez con `--packages`, veremos que la terminal empieza a descargar un montón de cosas. Es normal, está bajando los `.jar` de conexión).

Es necesario darle permisos a los Workers

1. Vamos a la consola web de AWS (EC2).
2. Seleccionamos las tres instancias Worker (`EC2-2_Spark_Worker1`, `EC2-3...`, `EC2-4...`). Se puede hacer de una en una.
3. Arriba a la derecha **Acciones > Seguridad > Modificar rol de IAM**.
4. Asignarles el rol **`LabInstanceProfile`** a las tres y guardar.

Actualizaremos el código para que Spark use ese rol

Volvemos a `EC2-5_Submit`, abrimos archivo con `nano job1_curation.py` y vamos a modificar solo la parte de arriba, cambiándola para que quede exactamente así (añadimos la línea del `.config`):

```
from pyspark.sql import SparkSession

from pyspark.sql.functions import col, to_date



spark = SparkSession.builder \

    .appName("Job1_Curation") \

    .config("spark.hadoop.fs.s3a.aws.credentials.provider",
"com.amazonaws.auth.DefaultAWSCredentialsProviderChain") \

    .getOrCreate()



ruta_base = "s3a://aurora-tickets-inigo/aurora/Inigo"



df_events = spark.read.option("header",
"true").csv(f"{ruta_base}/raw/events.csv")
```

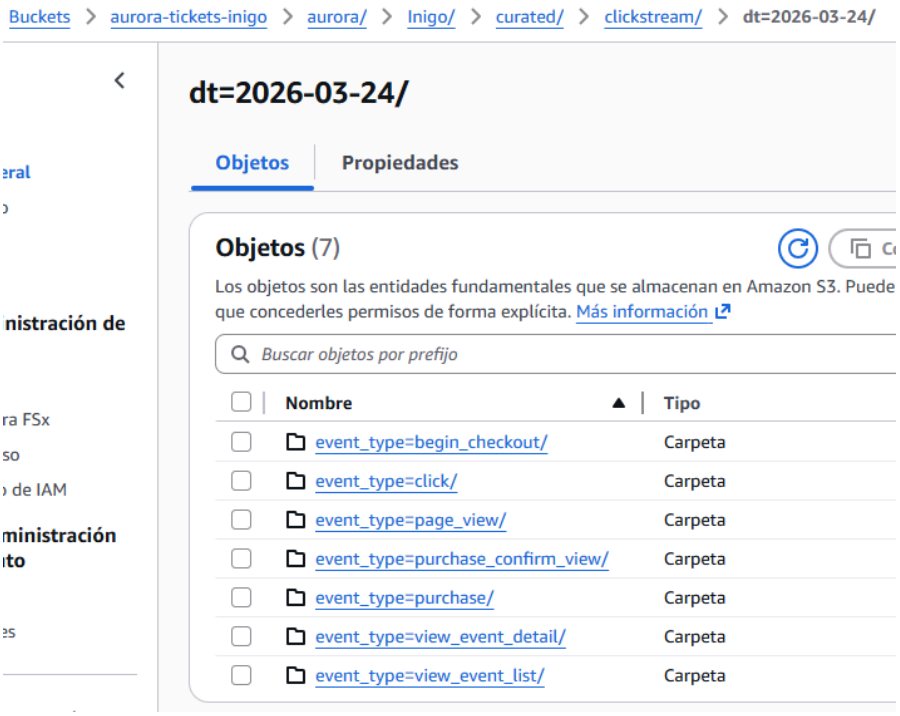
```
df_campaigns = spark.read.option("header",
"true").csv(f"{ruta_base}/raw/campaigns.csv")

df_transactions = spark.read.option("header",
"true").csv(f"{ruta_base}/raw/transactions.csv")

df_clickstream =
spark.read.json(f"{ruta_base}/raw/aurora_clickstream.jsonl")



df_events_clean = df_events.dropna(subset=["event_id"])



df_campaigns_clean =
df_campaigns.dropna(subset=["utm_campaign"])



df_transactions_clean =
df_transactions.dropna(subset=["transaction_id",
"session_id", "event_id"]) \

    .filter(col("amount") > 0) \

    .join(df_events_clean, "event_id", "left_semi")



df_clickstream_clean =
df_clickstream.dropna(subset=["timestamp", "session_id"]) \

    .withColumn("dt", to_date(col("timestamp"))) \

    .filter(col("dt").isNotNull())



df_events_clean.write.mode("overwrite").parquet(f"{ruta_base
}/curated/events")



df_campaigns_clean.write.mode("overwrite").parquet(f"{ruta_b
ase}/curated/campaigns")
```

```
df_transactions_clean.withColumn("dt",
to_date(col("timestamp"))) \


.write.mode("overwrite").partitionBy("dt").parquet(f"{ruta_b
ase}/curated/transactions")



df_clickstream_clean.write.mode("overwrite").partitionBy("dt
", "event_type").parquet(f"{ruta_base}/curated/clickstream")



spark.stop()
```

Guardamos el archivo (`Ctrl + O`, `Enter`, `Ctrl + X`) y volvemos a lanzar la bomba:

```
spark-submit --packages org.apache.hadoop:hadoop-aws:3.3.4
--master spark://172.31.14.249:7077 job1_curation.py
```

Entramos en la consola web de AWS y vamos a **S3**.
Navegamos por el bucket: `aurora-tickets-inigo` > `aurora` > `Inigo` > `curated` > `clickstream`.
Ahí dentro deberías ver unas carpetitas mágicas llamadas `dt=2026-03-...` y dentro de ellas el `event_type=...`.

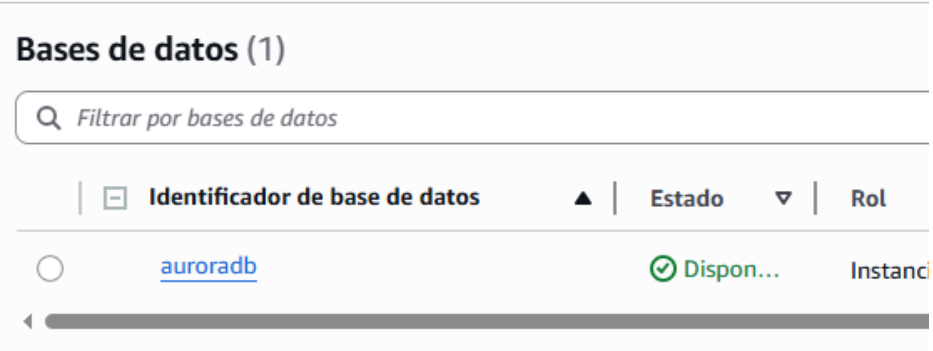


Creación de base de datos (RDS):

1. Vamos al servicio **RDS** y pulsamos **Crear base de datos**.
2. Seleccionamos **Creación estándar** y **MySQL**.
3. En Plantillas, **Capa gratuita** (si no deja, ponemos Desarrollo/Pruebas).
4. Identificador de base de datos: auroradb.
5. Nombre de usuario maestro: admin.
6. Contraseña principal: adminpassword (poner en los dos huecos).
7. En Conectividad, **Acceso público** en **Sí**.
8. Abrimos la sección **Configuración adicional** (suele estar escondida abajo del todo). En "Nombre de la base de datos inicial", escribimos aurora.
9. Pulsar **Crear base de datos**.

   Mientras AWS fabrica la base de datos, vamos a picar el código de Spark del **Job 2** en la máquina EC2-5_Submit.

   Este trabajo lee los datos limpios, calcula el funnel, el ranking y las anomalías, y los guarda en la carpeta analytics de S3 .

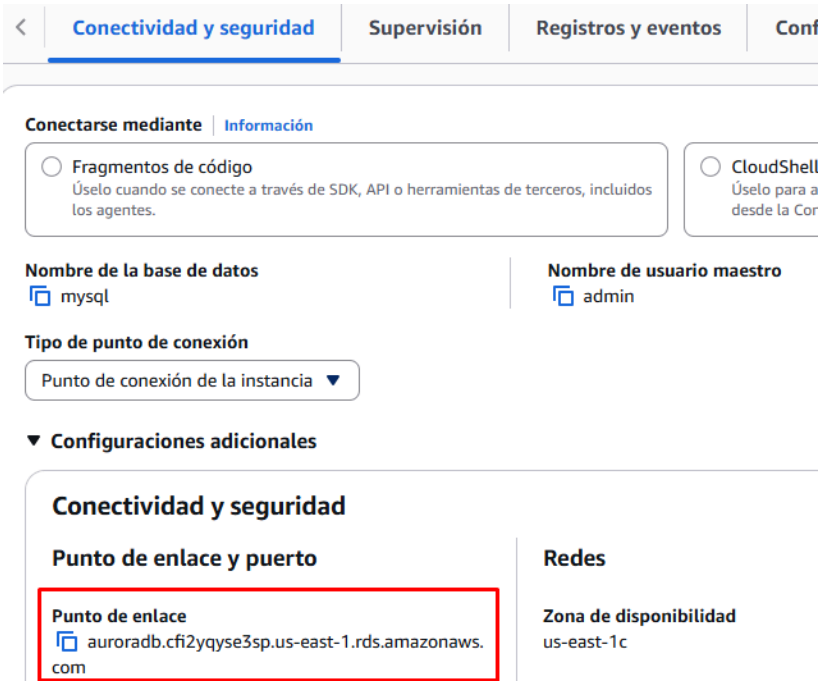


Job 2:

Sacar el Endpoint:

Ir a la consola de RDS, hacer clic en el nombre azul **`auroradb`**, en la pestaña "Conectividad y seguridad", hay un apartado que dice **Punto de enlace** (algo tipo `auroradb.xxxxxxx.us-east-1.rds.amazonaws.com`). Copiamos ese texto entero.

auroradb.cfi2yqyse3sp.us-east-1.rds.amazonaws.com



Crear la base de datos:

Volvemos a la terminal negra de la máquina `EC2-5_Submit`. Vamos a entrar a la base de datos usando el cliente de MySQL.

Lanzamos este comando, pero **sustituimos la parte de `PUNTO_DE_ENLACE`** por el punto de enlace real:

```
mysql -h PUNTO_DE_ENLACE -u admin -p
```

```
mysql -h auroradb.cfi2yqyse3sp.us-east-1.rds.amazonaws.com
-u admin -p
```

Nos pedirá la contraseña, escribimos `adminpassword` y le damos a Enter.

(Si dice `mysql: command not found`, es necesario instalar el cliente rápido con `sudo apt install mysql-client-core-8.0 -y` , como me pasó a mi y volvemos a probar).

```
ubuntu@ip-172-31-10-50:~$ mysql -h auroradb.cfi2yqyse3sp.us-east-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 37
Server version: 8.4.7 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql>
```

Cuando estemos dentro (se verá el texto `mysql>`), lanzando este comando crearemos la caja que nos falta:

```
CREATE DATABASE aurora;
```

Si dice `Query OK`, escribimos `exit` para salir.

```
Your MySQL connection id is 37
Server version: 8.4.7 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> CREATE DATABASE aurora;
Query OK, 1 row affected (0.02 sec)

mysql> exit
Bye
```

Actualizar el Job 2:

Abrimos el archivo en la máquina `EC2-5_Submit`:

```
nano job2_analytics.py
```

Borramos lo que había antes y pegamos este código final (le he añadido la conexión a RDS y la escritura de las tres tablas).

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, countDistinct, sum,
when, lit, count

spark = SparkSession.builder \
    .appName("Job2_Analytics") \
    .config("spark.hadoop.fs.s3a.aws.credentials.provider",
"com.amazonaws.auth.DefaultAWSCredentialsProviderChain") \
    .getOrCreate()

ruta_curated =
"s3a://aurora-tickets-inigo/aurora/Inigo/curated"
ruta_analytics =
"s3a://aurora-tickets-inigo/aurora/Inigo/analytics"
```

```
jdbc_url =
"jdbc:mysql://auroradb.cfi2yqyse3sp.us-east-1.rds.amazonaws.
com:3306/aurora"
propiedades_jdbc = {
    "user": "admin",
    "password": "adminpassword",
    "driver": "com.mysql.cj.jdbc.Driver"
}

df_clickstream =
spark.read.parquet(f"{ruta_curated}/clickstream")
df_transactions =
spark.read.parquet(f"{ruta_curated}/transactions")
df_events = spark.read.parquet(f"{ruta_curated}/events")

df_funnel = df_clickstream.groupBy("dt").agg(
    countDistinct("session_id").alias("sessions_total"),
    countDistinct(when(col("event_type") == "list",
col("session_id"))).alias("sessions_event_list"),
    countDistinct(when(col("event_type") == "detail",
col("session_id"))).alias("sessions_event_detail"),
    countDistinct(when(col("event_type") == "checkout",
col("session_id"))).alias("sessions_begin_checkout"),
    countDistinct(when(col("event_type") == "purchase",
col("session_id"))).alias("sessions_purchase")
).withColumn("conversion_rate", col("sessions_purchase") /
col("sessions_total"))

df_funnel.write.mode("overwrite").parquet(f"{ruta_analytics}
/funnel_daily")
df_funnel.write.jdbc(url=jdbc_url,
table="metrics_funnel_daily", mode="overwrite",
properties=propiedades_jdbc)

df_rank = df_clickstream.filter(col("event_type") ==
"detail").groupBy("dt", "event_id").agg(
    count("session_id").alias("detail_views")
).join(
    df_transactions.groupBy("dt", "event_id").agg(
        count("transaction_id").alias("purchases"),
        sum("amount").alias("revenue_total")
    ),
    ["dt", "event_id"],
```

```
    "left"
).fillna(0).withColumn("interest_to_purchase_ratio",
col("detail_views") / (col("purchases") + 1))

df_rank.write.mode("overwrite").parquet(f"{ruta_analytics}/e
vent_rank")
df_rank.write.jdbc(url=jdbc_url, table="metrics_event_rank",
mode="overwrite", properties=propiedades_jdbc)

df_anomalies = df_clickstream.groupBy("dt", "ip").agg(
    count("event_id").alias("requests")
).withColumn(
    "is_anomaly", when(col("requests") > 1000,
True).otherwise(False)
).withColumn(
    "reason", when(col("is_anomaly"), lit("Alta tasa de
peticiones")).otherwise(lit(""))
).filter(col("is_anomaly") == True)

df_anomalies.write.mode("overwrite").parquet(f"{ruta_analyti
cs}/anomalies")
df_anomalies.write.jdbc(url=jdbc_url,
table="metrics_anomalies", mode="overwrite",
properties=propiedades_jdbc)

spark.stop()
```

Guardarlo con `Ctrl + O`, `Enter` y `Ctrl + X`.

Lanzar el Job 2 al clúster:

Para qué Spark pueda hablar con MySQL, le tenemos que dar también la librería de MySQL junto a la de AWS que ya usamos antes.

Lanzamos este comando en la terminal `EC2-5_Submit`:

```
spark-submit --packages
org.apache.hadoop:hadoop-aws:3.3.4,mysql:mysql-connector-jav
a:8.0.33 --master spark://172.31.14.249:7077
job2_analytics.py
```

```
26/03/25 18:12:43 INFO DAGScheduler: running: Set()
26/03/25 18:12:43 INFO DAGScheduler: waiting: Set()
26/03/25 18:12:43 INFO DAGScheduler: failed: Set()
26/03/25 18:12:43 INFO ShufflePartitionsUtil: For shuffle(7), advisory target size: 67108864, actual target size 1048576, minimum partition size: 1048576
26/03/25 18:12:43 INFO HashAggregateExec: spark.sql.codegen.aggregate.map.twolevel.enabled is set to true, but current version of codegened fast hashmap does not support this aggregate.
26/03/25 18:12:43 INFO SparkContext: Starting job: jdbc at NativeMethodAccessorImpl.java:0
26/03/25 18:12:43 INFO DAGScheduler: Got job 16 (jdbc at NativeMethodAccessorImpl.java:0) with 1 output partitions
26/03/25 18:12:43 INFO DAGScheduler: Final stage: ResultStage 23 (jdbc at NativeMethodAccessorImpl.java:0)
26/03/25 18:12:43 INFO DAGScheduler: Parents of final stage: List(ShuffleMapStage 22)
26/03/25 18:12:43 INFO DAGScheduler: Missing parents: List()
26/03/25 18:12:43 INFO DAGScheduler: Submitting ResultStage 23 (MapPartitionsRDD[66] at jdbc at NativeMethodAccessorImpl.java:0), which has no missing parents
26/03/25 18:12:43 INFO MemoryStore: Block broadcast_23 stored as values in memory (estimated size 54.3 KiB, free 434.1 MiB)
26/03/25 18:12:43 INFO MemoryStore: Block broadcast_23_piece0 stored as bytes in memory (estimated size 24.0 KiB, free 434.0 MiB)
26/03/25 18:12:43 INFO BlockManagerInfo: Added broadcast_23_piece0 in memory on ip-172-31-10-50.ec2.internal:34009 (size: 24.0 KiB, free: 434.3 MiB)
26/03/25 18:12:43 INFO BlockManagerInfo: Removed broadcast_22_piece0 on ip-172-31-10-50.ec2.internal:34009 in memory (size: 18.7 KiB, free: 434.3 MiB)
26/03/25 18:12:43 INFO BlockManagerInfo: Removed broadcast_22_piece0 on 172.31.1.220:42577 in memory (size: 18.7 KiB, free: 434.4 MiB)
26/03/25 18:12:43 INFO SparkContext: Created broadcast 23 from broadcast at DAGScheduler.scala:1585
26/03/25 18:12:43 INFO BlockManagerInfo: Removed broadcast_22_piece0 on 172.31.3.18:40617 in memory (size: 18.7 KiB, free: 434.4 MiB)
26/03/25 18:12:43 INFO DAGScheduler: Submitting 1 missing tasks from ResultStage 23 (MapPartitionsRDD[66] at jdbc at NativeMethodAccessorImpl.java:0) (first 15 tasks are for partitions Vector(0))
26/03/25 18:12:43 INFO TaskSchedulerImpl: Adding task set 23.0 with 1 tasks resource profile 0
26/03/25 18:12:43 INFO BlockManagerInfo: Removed broadcast_22_piece0 on 172.31.11.14:37129 in memory (size: 18.7 KiB, free: 434.4 MiB)
26/03/25 18:12:43 INFO TaskSetManager: Starting task 0.0 in stage 23.0 (TID 43) (172.31.3.18, executor 0, partition 0, NODE_LOCAL, 8825 bytes)
26/03/25 18:12:43 INFO BlockManagerInfo: Added broadcast_23_piece0 in memory on 172.31.3.18:40617 (size: 24.0 KiB, free: 434.3 MiB)
26/03/25 18:12:43 INFO MapOutputTrackerMasterEndpoint: Asked to send map output locations for shuffle 7 to 172.31.3.18:37686
26/03/25 18:12:45 INFO TaskSetManager: Finished task 0.0 in stage 23.0 (TID 43) in 1603 ms on 172.31.3.18 (executor 0) (1/1)
26/03/25 18:12:45 INFO TaskSchedulerImpl: Removed TaskSet 23.0, whose tasks have all completed, from pool
26/03/25 18:12:45 INFO DAGScheduler: ResultStage 23 (jdbc at NativeMethodAccessorImpl.java:0) finished in 1.643 s
26/03/25 18:12:45 INFO DAGScheduler: Job 16 is finished. Cancelling potential speculative or zombie tasks for this job
26/03/25 18:12:45 INFO TaskSchedulerImpl: Killing all running tasks in stage 23: Stage finished
26/03/25 18:12:45 INFO DAGScheduler: Job 16 finished: jdbc at NativeMethodAccessorImpl.java:0, took 1.653831 s
26/03/25 18:12:45 INFO SparkContext: SparkContext is stopping with exitCode 0.
26/03/25 18:12:45 INFO SparkUI: Stopped Spark web UI at http://ip-172-31-10-50.ec2.internal:4040
26/03/25 18:12:45 INFO StandaloneSchedulerBackend: Shutting down all executors
26/03/25 18:12:45 INFO StandaloneSchedulerBackend$StandaloneDriverEndpoint: Asking each executor to shut down
26/03/25 18:12:45 INFO MapOutputTrackerMasterEndpoint: MapOutputTrackerMasterEndpoint stopped!
26/03/25 18:12:45 INFO MemoryStore: MemoryStore cleared
26/03/25 18:12:45 INFO BlockManager: BlockManager stopped
26/03/25 18:12:45 INFO BlockManagerMaster: BlockManagerMaster stopped
26/03/25 18:12:45 INFO OutputCommitCoordinator$OutputCommitCoordinatorEndpoint: OutputCommitCoordinator stopped!
26/03/25 18:12:45 INFO SparkContext: Successfully stopped SparkContext
26/03/25 18:12:45 INFO ShutdownHookManager: Shutdown hook called
26/03/25 18:12:45 INFO ShutdownHookManager: Deleting directory /tmp/spark-26400f21-5a99-4656-bd8a-915482c13f6f
26/03/25 18:12:45 INFO ShutdownHookManager: Deleting directory /tmp/spark-3a983004-1d07-4861-9fa7-b73f4b906dd7
26/03/25 18:12:45 INFO ShutdownHookManager: Deleting directory /tmp/spark-26400f21-5a99-4656-bd8a-915482c13f6f/pyspark-e8210534-4d1c-487e-8bcd-05065be8e130
26/03/25 18:12:45 INFO MetricsSystemImpl: Stopping s3a-file-system metrics system...
26/03/25 18:12:45 INFO MetricsSystemImpl: s3a-file-system metrics system stopped.
26/03/25 18:12:45 INFO MetricsSystemImpl: s3a-file-system metrics system shutdown complete.
ubuntu@ip-172-31-10-50:~$
```

Spark ha triturado los datos, calculado los KPIs y los ha mandado directos a S3 y a la nueva base de datos MySQL.

Vamos a entrar a la caja fuerte a ver qué ha metido Spark.

1. Nos volvemos a conectar a la base de datos desde la terminal con el comando de antes (la contraseña era `adminpassword`):

```
mysql -h auroradb.cfi2yqyse3sp.us-east-1.rds.amazonaws.com
-u admin -p
```

Una vez dentro (cuando veamos el `mysql>`), usar la base de datos que creamos antes:

```
USE aurora;
```

Vamos a ver si Spark ha creado las tres tablas de métricas:

```
SHOW TABLES;
```

Ahora vamos a observar los datos del funnel de conversión.
Lanzamos esto y demostraremos que hay datos dentro:

```
SELECT * FROM metrics_funnel_daily;
```

Aurora Tickets

```
ubuntu@ip-172-31-10-50:~$ mysql -h auroradb.cfi2yqyse3sp.us-east-1.rds.amazonaws.com -u admin -p
Enter password:
Welcome to the MySQL monitor.  Commands end with ; or \g.
Your MySQL connection id is 65
Server version: 8.4.7 Source distribution

Copyright (c) 2000, 2026, Oracle and/or its affiliates.

Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.

Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.

mysql> USE aurora;
Reading table information for completion of table and column names
You can turn off this feature to get a quicker startup with -A

Database changed
mysql> SHOW TABLES;
+----------------------+
| Tables_in_aurora     |
+----------------------+
| metrics_anomalies    |
| metrics_event_rank   |
| metrics_funnel_daily |
+----------------------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM metrics_funnel_daily;
+------------+----------------+--------------------+----------------------+-------------------------+-------------------+--------------------+
| dt         | sessions_total | sessions_event_list | sessions_event_detail | sessions_begin_checkout | sessions_purchase | conversion_rate    |
+------------+----------------+--------------------+----------------------+-------------------------+-------------------+--------------------+
| 2026-03-24 |          30001 |                  0 |                    0 |                       0 |             15024 | 0.5007833072230926 |
+------------+----------------+--------------------+----------------------+-------------------------+-------------------+--------------------+
1 row in set (0.00 sec)
```

```
mysql> SELECT * FROM metrics_funnel_daily;
+------------+----------------+--------------------+----------------------+-------------------------+-------------------+--------------------+
| dt         | sessions_total | sessions_event_list | sessions_event_detail | sessions_begin_checkout | sessions_purchase | conversion_rate    |
+------------+----------------+--------------------+----------------------+-------------------------+-------------------+--------------------+
| 2026-03-24 |          30001 |                  0 |                    0 |                       0 |             15024 | 0.5007833072230926 |
+------------+----------------+--------------------+----------------------+-------------------------+-------------------+--------------------+
1 row in set (0.00 sec)
```

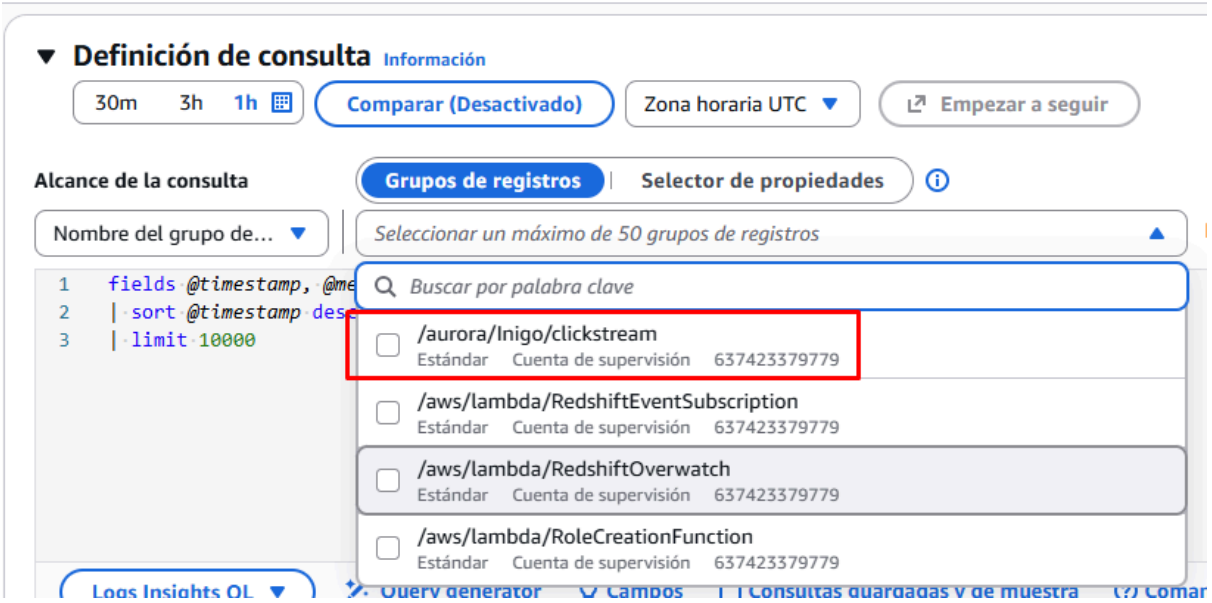
```
Type 'help;' or '\h' for help. Ty

mysql> USE aurora;
Reading table information for con
You can turn off this feature to

Database changed
mysql> SHOW TABLES;
+----------------------+
| Tables_in_aurora     |
+----------------------+
| metrics_anomalies    |
| metrics_event_rank   |
| metrics_funnel_daily |
+----------------------+
3 rows in set (0.00 sec)
```

7. CloudWatch (Insights y Dashboards)

Esta es la parte más visual: sacar métricas en tiempo real con **CloudWatch**.

Vámonos a la consola web de AWS:

1. Buscamos el servicio **CloudWatch** en el buscador de AWS.
2. En el menú de la izquierda, desplegamos la sección **Registros** (Logs) y hacemos clic en **Información sobre registros** (Logs Insights).
3. Arriba, donde pone "Seleccionar grupo de registros", tenemos que buscar y marcar el grupo de la máquina web (EC2-6).



6.2 Logs Insights

Debes crear **y guardar** al menos **4 consultas** con nombres estandarizados:

1. `aurora_q1_funnel_daily`
2. `aurora_q2_top_events_interest_vs_revenue`
3. `aurora_q3_errors_and_latency`
4. `aurora_q4_anomalies_suspected_bots`

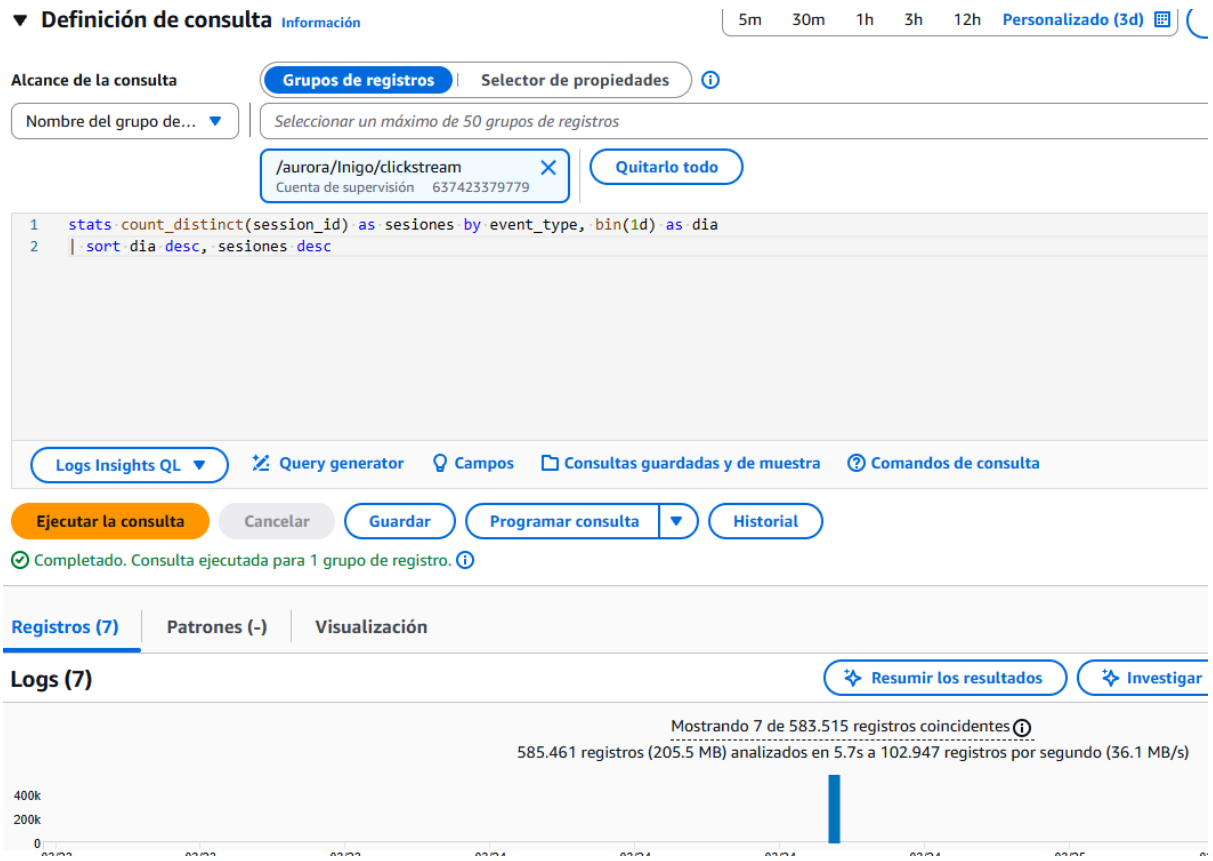
No se evalúa solo que "funcionen"; se evalúa que respondan a preguntas de negocio con sentido.

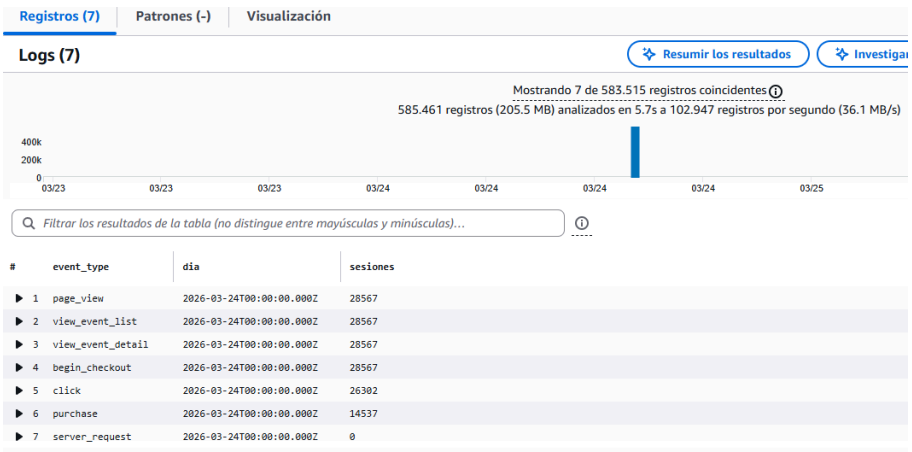
Aurora Tickets

1. aurora_q1_funnel_daily:

Mide cuántas sesiones únicas pasan por cada fase del embudo al día.

```
stats count_distinct(session_id) as sesiones by event_type,
bin(1d) as dia

| sort dia desc, sesiones desc
```





#	event_type	dia	sesiones
1	page_view	2026-03-24T00:00:00.000Z	28567
2	view_event_list	2026-03-24T00:00:00.000Z	28567
3	view_event_detail	2026-03-24T00:00:00.000Z	28567
4	begin_checkout	2026-03-24T00:00:00.000Z	28567
5	click	2026-03-24T00:00:00.000Z	26302
6	purchase	2026-03-24T00:00:00.000Z	14537
7	server_request	2026-03-24T00:00:00.000Z	0

2. aurora_q2_top_events_interest_vs_revenue:

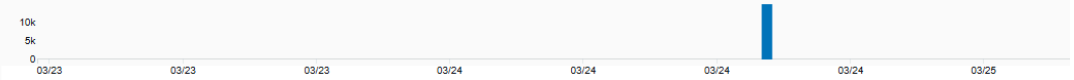
Muestra los eventos (entradas) que generan más interés (vistas) y ventas.

```
filter event_type in ["detail", "purchase"]

| stats count(*) as total by event_id, event_type

| sort total desc

| limit 20
```

Registros (20) | Patrones (-) | Visualización

Logs (20) — Resumir los resultados

Mostrando 20 de 14.977 registros coincidentes
585.461 registros (205.5 MB) analizados en 1.5s a 389.269 registros por segundo (136.6 MB/s)



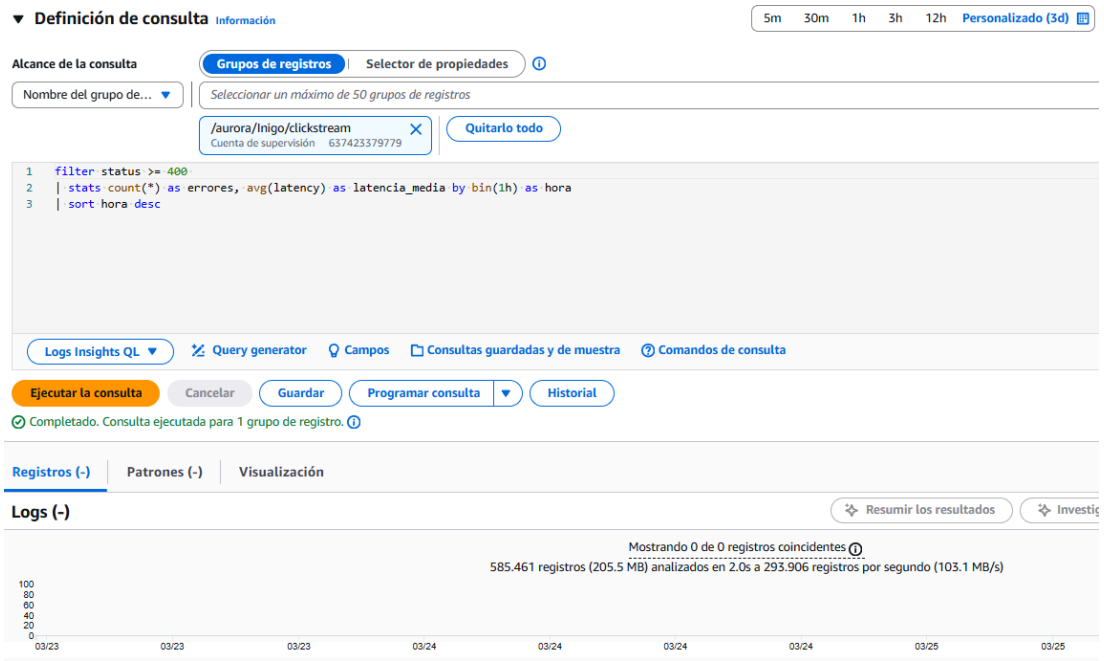
Filtrar los resultados de la tabla (no distingue entre mayúsculas y minúsculas)...

#	event_id	event_type	total
1	61305061	purchase	176
2	61305003	purchase	171
3	61305025	purchase	168
4	61305101	purchase	162
5	61305105	purchase	162
6	61305045	purchase	162
7	61305096	purchase	160
8	61305084	purchase	157
9	61305011	purchase	157
10	61305058	purchase	157
11	61305094	purchase	156
12	61305049	purchase	156
13	61305083	purchase	156
14	61305067	purchase	155
15	61305047	purchase	154
16	61305056	purchase	152
17	61305002	purchase	151
18	61305031	purchase	150
19	61305062	purchase	150
20	61305037	purchase	149

3. aurora_q3_errors_and_latency:

Busca fallos en la web y calcula el tiempo de respuesta medio.

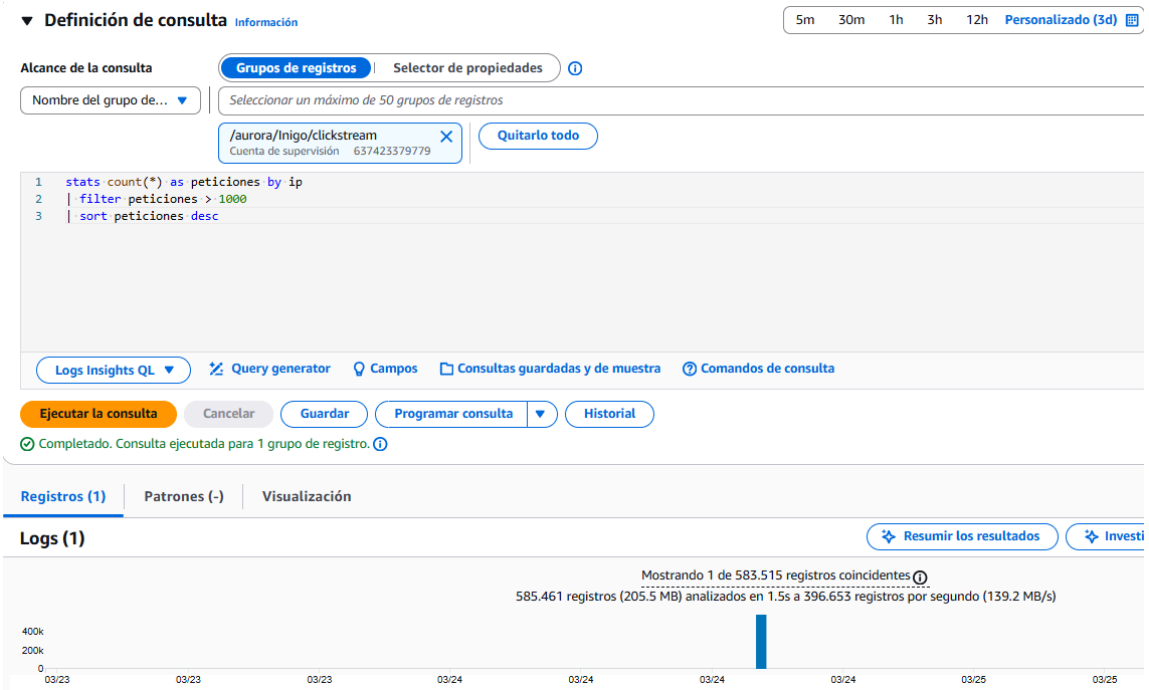
```
filter status >= 400
| stats count(*) as errores, avg(latency) as latencia_media
by bin(1h) as hora
| sort hora desc
```



4. aurora_q4_anomalies_suspected_bots:

Detecta direcciones IP con un número sospechoso de peticiones.

```
stats count(*) as peticiones by ip
| filter peticiones > 1000
| sort peticiones desc
```



Guardaremos todas estas consultas para luego importarlas fácilmente en el dashboard.

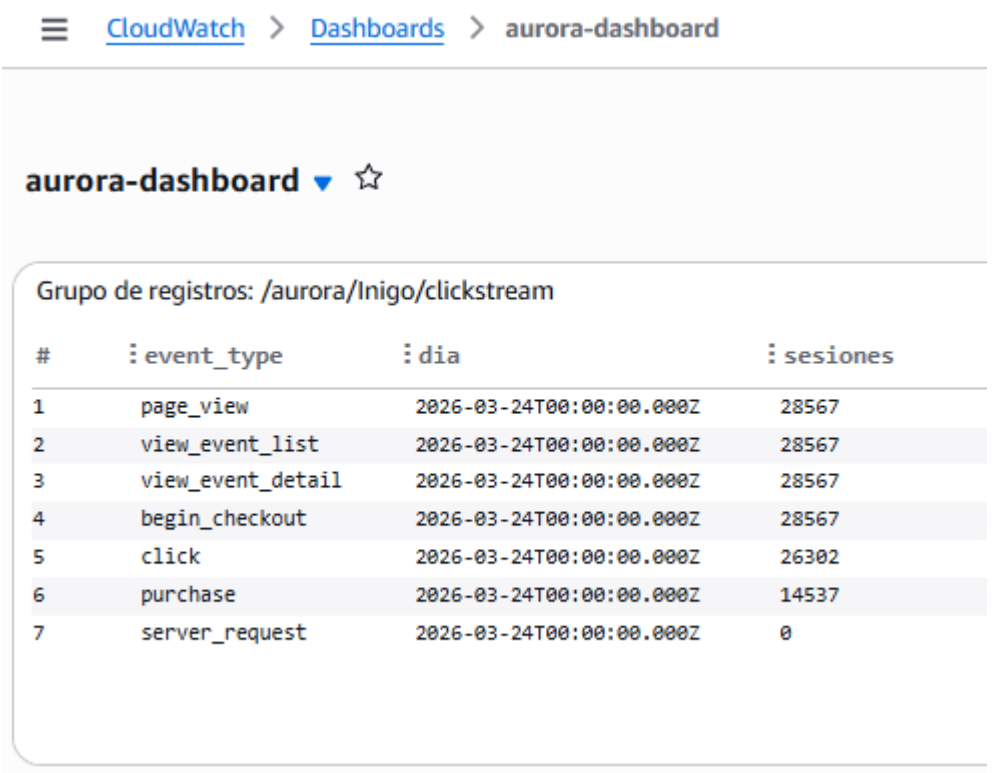
Panel de control (Dashboard):

Esto es lo que vería el "cliente" final para analizar su negocio.

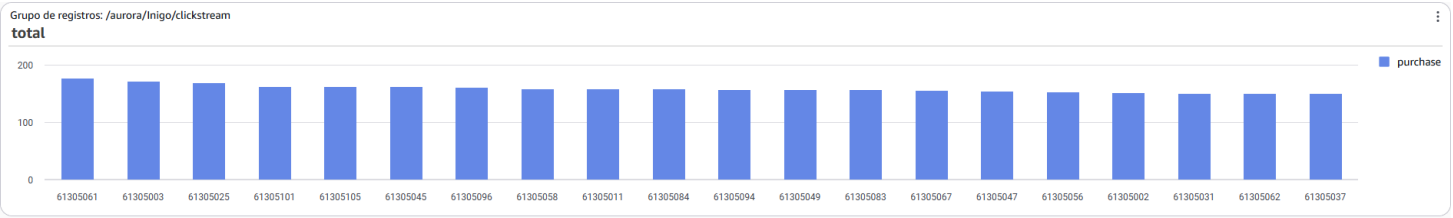
Seguimos estos pasos en la consola de AWS:

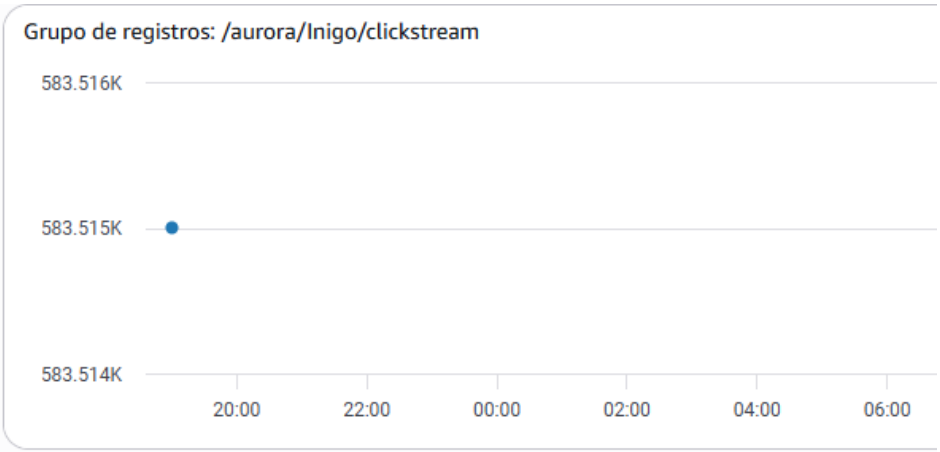
1. En el menú izquierdo de CloudWatch, hacemos clic en **Paneles** (Dashboards).
2. Pulsamos el botón naranja **Crear panel**.
3. Ponemos un nombre: `aurora-dashboard`.
4. Nos pedirá añadir el primer widget., elegimos un tipo de gráfico (**línea** para el tráfico o **tabla de registros** para el top de IPs).
5. Seleccionar **Registros** (Logs) como origen de datos.
6. Elegir el grupo de registros de tu máquina web (el mismo que usaste en las consultas).
7. Abajo, en la pestaña **Consultas guardadas**, seleccionar la primera consulta (`aurora_q1_funnel_daily`).

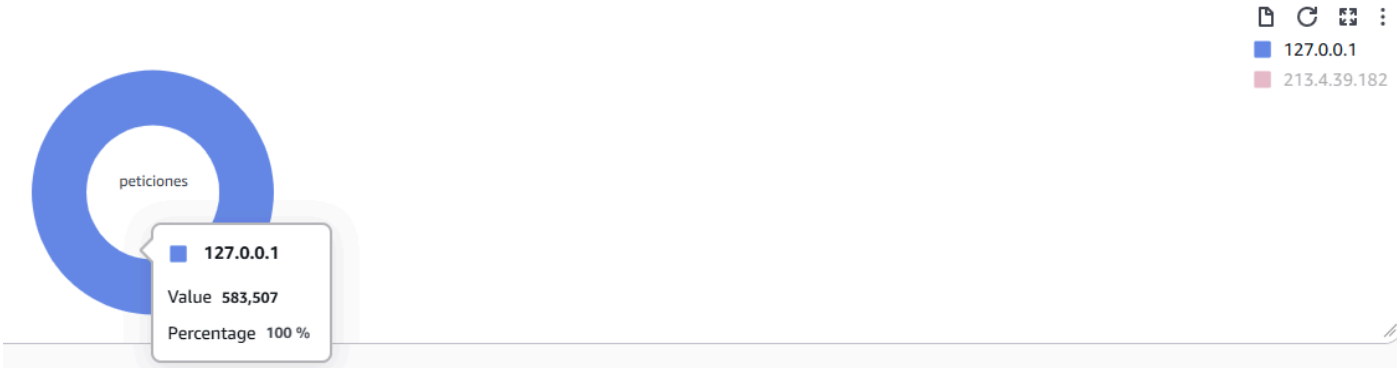
8. Pulsar **Crear widget**.



#	event_type	dia	sesiones
1	page_view	2026-03-24T00:00:00.000Z	28567
2	view_event_list	2026-03-24T00:00:00.000Z	28567
3	view_event_detail	2026-03-24T00:00:00.000Z	28567
4	begin_checkout	2026-03-24T00:00:00.000Z	28567
5	click	2026-03-24T00:00:00.000Z	26302
6	purchase	2026-03-24T00:00:00.000Z	14537
7	server_request	2026-03-24T00:00:00.000Z	0







8. COSTES

El proyecto utiliza un modelo de pago por uso, esta es la estimación mensual si la infraestructura estuviera encendida 24 horas al día:

- Amazon EC2 (6 instancias t3.medium): 50$ aprox.
- Amazon RDS (MySQL db.t3.medium): 15$ approx.
- Amazon S3 y CloudWatch: Menos de 1$.
- Coste total estimado: 66$ al mes.
- Coste real del proyecto: Al usar el entorno de AWS Academy, los recursos se apagan tras la ejecución. El coste real de esta práctica ha sido de apenas unos 4$.